

ABRAHAM OF LONDON

THE ENGINEERING MANUAL

Decision Authority Infrastructure — Technical Reference

Document Control

Field	Detail
Classification	Internal — Engineering
Version	1.0
Effective Date	April 2026
Review Cycle	Quarterly
Custodian	Lead Engineer
Next Review	July 2026

PREAMBLE

This document constitutes the definitive technical reference for all engineering work on the Abraham of London platform. It governs how the system is built, deployed, maintained, and extended. Where the Institutional Manual defines what the platform does — its purpose, its products, its commercial doctrine — this Engineering Manual defines the architecture, patterns, constraints, and standards that make that purpose executable in code.

Every line of code committed to this repository is subject to the standards contained herein. This is not a set of suggestions or best practices to be weighed against developer preference. It is doctrine. The platform's reliability, security, and capacity to scale depend on uniform adherence to these patterns. Deviations require written justification and approval from the Lead Engineer.

This manual is a living document, reviewed quarterly and updated as the platform evolves. However, its core architectural principles — separation of concerns, type safety, server-first rendering, content-as-data — are permanent. They will outlast any individual framework version. When upgrading dependencies or adopting new tools, these principles take precedence over convenience.

HOW TO USE THIS MANUAL

Role-Based Reading Paths

Role	Required Reading
New Developer	Parts I–III (Architecture, Core Systems, Data Layer)
Frontend Engineer	Parts II, IV (Core Systems, UI and Presentation)
Backend Engineer	Parts II, III, V (Core Systems, Data, APIs and Integrations)

DevOps / Platform	Parts VI–VII (Deployment, Monitoring and Operations)
Full-Stack Engineer	All Parts sequentially

Conventions Used in This Document

Convention	Meaning
<code>monospace</code>	File paths, commands, code identifiers
Bold	Critical terms, model names, required values
→	Cross-reference to another section
// ...	Omitted code for brevity
Key Principle block	Foundational rule that must never be violated
Warning block	Common mistake or dangerous anti-pattern

KEY PRINCIPLE

When this document conflicts with external documentation (framework docs, blog posts, Stack Overflow), this document takes precedence for this codebase. External patterns are adopted only after validation against the architectural principles defined in Part I.

TABLE OF CONTENTS

Part I — System Architecture

- Chapter 1: Platform Overview
 - 1.1 What the System Is
 - 1.2 High-Level Architecture
 - 1.3 The Five Layers
 - 1.4 Directory Structure
 - 1.5 Module Boundaries
- Chapter 2: Technology Stack
 - 2.1 Complete Dependency Catalog
 - 2.2 Version Pinning Policy
 - 2.3 Upgrade Strategy
 - 2.4 Compatibility Matrix
 - 2.5 Prohibited Dependencies
- Chapter 3: Application Architecture
 - 3.1 Next.js App Router
 - 3.2 Legacy Pages Router
 - 3.3 Rendering Strategies
 - 3.4 Provider Composition
 - 3.5 Server Components vs Client Components
 - 3.6 Server Actions
 - 3.7 Middleware

Part II — Core Systems

- Chapter 4: Database Layer
 - 4.1 Prisma Architecture
 - 4.2 Schema Overview — Domain Groupings
 - 4.3 Key Enumerations
 - 4.4 Connection Patterns
 - 4.5 Migration Strategy
 - 4.6 Commands Reference
 - 4.7 Seeding
 - 4.8 Server-Only Import Rule
- Chapter 5: Authentication and Authorization
 - 5.1 NextAuth Configuration
 - 5.2 OAuth Flow
 - 5.3 Credentials Flow
 - 5.4 JWT Structure and Claims
 - 5.5 Session Access Patterns
 - 5.6 Role System
 - 5.7 Access Tier Hierarchy
 - 5.8 Guard Functions
 - 5.9 Cookie Configuration
 - 5.10 Multi-Factor Authentication
 - 5.11 Admin Bootstrap
- Chapter 6: Content System
 - 6.1 Contentlayer2 Architecture
 - 6.2 Document Type Definitions
 - 6.3 Frontmatter Schema
 - 6.4 Processing Pipeline
 - 6.5 Remark Plugins
 - 6.6 Rehye Plugins
 - 6.7 Generated Types Usage
 - 6.8 Content Routing
 - 6.9 ISR and Cache Strategy
 - 6.10 Windows Compatibility

Part III — Data and State

- Chapter 7: State Management
- Chapter 8: Data Fetching Patterns
- Chapter 9: Caching Architecture

Part IV — UI and Presentation

- Chapter 10: Styling System
- Chapter 11: Component Architecture
- Chapter 12: Design Tokens and Theme

Part V — APIs and Integrations

- Chapter 13: API Design
- Chapter 14: Email System
- Chapter 15: Payment Processing
- Chapter 16: AI Integration
- Chapter 17: Search

- Chapter 18: PDF Generation

Part VI — Deployment and Infrastructure

- Chapter 19: Build Pipeline
- Chapter 20: Environment Configuration
- Chapter 21: Netlify Deployment
- Chapter 22: Docker Configuration

Part VII — Quality and Operations

- Chapter 23: Testing Strategy
- Chapter 24: Security Hardening
- Chapter 25: Monitoring and Observability
- Chapter 26: Incident Response

Appendices

- Appendix A: Environment Variables Reference
- Appendix B: CLI Commands Quick Reference
- Appendix C: Troubleshooting Guide
- Appendix D: Architecture Decision Records

PART I — SYSTEM ARCHITECTURE

Chapter 1: Platform Overview

1.1 What the System Is

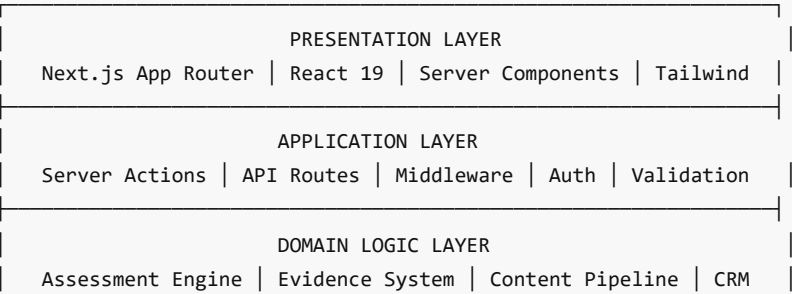
Abraham of London is a **Decision Authority Infrastructure** platform. It is a full-stack web application that delivers structured decision-forcing interventions to enterprise clients through assessments, strategy sessions, evidence systems, and content delivery. The platform serves both the public-facing marketing and content surface and the authenticated client delivery infrastructure.

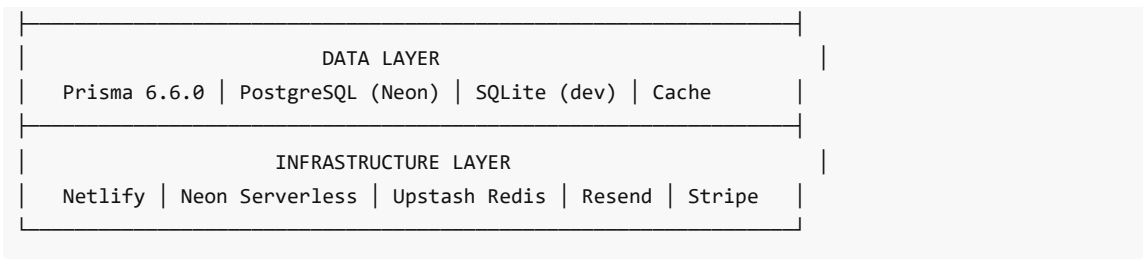
The system is not a blog. It is not a SaaS dashboard. It is a commercial delivery mechanism with multiple revenue surfaces — from free content that triggers engagement, through paid assessments and strategy rooms, to enterprise retainer management. Every technical decision serves this commercial architecture.

KEY PRINCIPLE

The platform exists to convert attention into revenue through structured confrontation. Every feature, every route, every component must trace its justification back to a stage in the product ladder defined in the Institutional Manual, Part II.

1.2 High-Level Architecture





The architecture follows a strict top-down dependency rule: each layer may only depend on the layer immediately below it. The Presentation layer never touches the database directly. The Data layer never imports React components. Violations of this layering are treated as architectural defects.

1.3 The Five Layers

Layer	Responsibility	Key Technologies	Location
Presentation	UI rendering, user interaction, visual state	React 19, Tailwind, Radix UI, CVA	app/, components/
Application	Request handling, orchestration, auth enforcement	Next.js middleware, Server Actions, API routes	app/actions/, pages/api/, middleware.ts
Domain Logic	Business rules, assessment scoring, content processing	Pure TypeScript modules	lib/, services/
Data	Persistence, queries, schema, migrations	Prisma, PostgreSQL, SQLite	prisma/, lib/prisma.server.ts
Infrastructure	Hosting, external services, network	Netlify, Neon, Upstash, Resend, Stripe	netlify.toml, env vars

⚠ WARNING

Importing `@prisma/client` in any file that runs in the browser is a critical violation. Prisma is server-only. All database access must route through `Lib/prisma.server.ts` and be consumed exclusively in Server Components, Server Actions, or API routes.

1.4 Directory Structure

C:\aol-check-visual\

└─ app/

├─ (public)/

├─ (authenticated)/

├─ (admin)/

├─ actions/

├─ api/

└─ layout.tsx

└─ pages/

└─ api/

└─ components/

├─ ui/

├─ layout/

└─ features/

→ App Router: routes, layouts, pages, loading/error states

→ Public route group (no auth required)

→ Protected route group (auth required)

→ Admin route group (ADMIN/OWNER role required)

→ Server Actions (form handlers, mutations)

→ App Router API routes (preferred for new endpoints)

→ Root layout with provider composition

→ Legacy Pages Router (auth callbacks, debug endpoints)

→ Pages Router API routes (legacy, do not add new)

→ Shared UI components

→ Primitives (Button, Card, Input, Dialog)

→ Layout components (Header, Footer, Sidebar)

→ Feature-specific composed components

└─ lib/	→ Domain logic, utilities, service clients
└─ prisma.server.ts	→ Database singleton (SERVER ONLY)
└─ auth/	→ Auth utilities, guards, session helpers
└─ ai/	→ AI integration (Anthropic, OpenAI)
└─ email/	→ Email templates and sending logic
└─ stripe/	→ Payment processing
└─ validation/	→ Zod schemas
└─ prisma/	→ Database schema, migrations, seed
└─ schema.prisma	→ The single source of truth for data models
└─ migrations/	→ Production migration history
└─ seed.ts	→ Development data seeding
└─ content/	→ MDX content files (processed by Contentlayer2)
└─ briefs/	→ Decision briefs
└─ blog/	→ Blog posts
└─ vault/	→ Premium gated content
└─ registry/	→ Institutional registry entries
└─ public/	→ Static assets (images, fonts, favicons)
└─ styles/	→ Global CSS, Tailwind base layer
└─ types/	→ Shared TypeScript type definitions
└─ scripts/	→ Build scripts, content processing, utilities
└─ _templates/	→ Code generation templates
└─ _tests_/	→ Test suites (Vitest unit, Playwright E2E)
└─ middleware.ts	→ Edge middleware (auth, redirects, rate limiting)
└─ contentlayer.config.ts	→ Contentlayer2 document type definitions
└─ tailwind.config.ts	→ Tailwind configuration with design tokens
└─ next.config.ts	→ Next.js configuration
└─ tsconfig.json	→ TypeScript strict configuration
└─ package.json	→ Dependencies and scripts

KEY PRINCIPLE

New files must be placed according to this structure without exception. If a file does not have an obvious home, it belongs in `lib/` with a descriptive filename. Creating new top-level directories requires Lead Engineer approval.

1.5 Module Boundaries

The following import rules are enforced:

Source	May Import From	Must Never Import From
app/	components/, lib/, types/	prisma/ directly, node_modules internals
components/	Other components/, lib/, types/	app/, prisma/, pages/
lib/	Other lib/, types/, external packages	app/, components/, pages/
pages/api/	lib/, types/, prisma/ (via server module)	app/, components/
prisma/	Nothing (leaf module)	Everything else

Chapter 2: Technology Stack

2.1 Complete Dependency Catalog

Runtime Framework

Package	Version	Purpose
next	16.2.1	Full-stack React framework, App Router, API routes
react	19.2.4	UI library, Server Components, Suspense
react-dom	19.2.4	DOM rendering
typescript	5.9.3	Type system (strict mode enforced)

Database and ORM

Package	Version	Purpose
prisma	6.6.0	Schema definition, migrations, CLI
@prisma/client	6.6.0	Type-safe database client
@neondatabase/serverless	—	Serverless PostgreSQL driver (production)
better-sqlite3	—	SQLite driver (development)

Authentication

Package	Version	Purpose
next-auth	4.24.13	Authentication framework
@auth/prisma-adapter	—	Prisma session/account storage
argon2	—	Password hashing (Credentials provider)
otplib	—	TOTP generation and verification (MFA)

Styling and UI

Package	Version	Purpose
tailwindcss	3.4.17	Utility-first CSS framework
@radix-ui/*	—	Accessible headless UI primitives
class-variance-authority	—	Component variant management (CVA)
clsx + tailwind-merge	—	Conditional class composition
@emotion/react	—	CSS-in-JS fallback (legacy components)

State Management

Package	Version	Purpose
zustand	5.0.12	Primary client state (lightweight stores)

@reduxjs/toolkit	—	Secondary state (complex workflows)
@tanstack/react-query	—	Server state, caching, revalidation

Content Processing

Package	Version	Purpose
contentlayer2	—	MDX processing, type generation
mdx	—	Markdown + JSX authoring
gray-matter	—	Frontmatter parsing
remark-gfm	—	GitHub Flavored Markdown
rehype-pretty-code	—	Syntax highlighting
rehype-slug	—	Heading anchors
rehype-autolink-headings	—	Clickable heading links

Email

Package	Version	Purpose
resend	6.9.3	Primary email delivery
nodemailer	—	Fallback email transport

PDF Generation

Package	Version	Purpose
@react-pdf/renderer	—	React-based PDF composition
jspdf	—	Programmatic PDF generation
puppeteer	—	HTML-to-PDF via headless Chrome
md-to-pdf	—	Markdown-to-PDF conversion

AI Integration

Package	Version	Purpose
@anthropic-ai/sdk	—	Claude API (primary AI provider)
openai	—	OpenAI API (secondary)

Payments

Package	Version	Purpose
stripe	—	Payment processing, subscriptions, webhooks

Search

Package	Version	Purpose
algoliasearch	—	Full-text search (production)
fuse.js	—	Client-side fuzzy search (fallback)

Security

Package	Version	Purpose
@upstash/ratelimit	—	Distributed rate limiting
zod	—	Runtime schema validation
dompurify	—	HTML sanitization
csrf	—	CSRF token generation and verification

Monitoring

Package	Version	Purpose
@vercel/analytics	—	Page view and event analytics
@vercel/speed-insights	—	Core Web Vitals monitoring

Testing

Package	Version	Purpose
vitest	4.1.2	Unit and integration testing
@playwright/test	—	End-to-end browser testing

2.2 Version Pinning Policy

All dependencies use **exact versions** in `package.json` (no `^` or `~` prefixes). This eliminates supply-chain drift and ensures reproducible builds across all environments.

```
{
  "dependencies": {
    "next": "16.2.1",      // ✓ Exact
    "react": "19.2.4",    // ✓ Exact
    "next": "^16.2.1"     // ✗ PROHIBITED – semver range
  }
}
```

⚠ WARNING

Running `pnpm update` without explicit package names is prohibited. Bulk updates bypass review and can introduce breaking changes. Every dependency upgrade must be individually justified, tested, and committed separately.

2.3 Upgrade Strategy

- 1. **Patch versions** (e.g., 16.2.1 → 16.2.2) — Apply after verifying changelog. No approval required.
- 2. **Minor versions** (e.g., 16.2.x → 16.3.0) — Apply in dedicated branch. Run full test suite. Requires code review.
- 3. **Major versions** (e.g., 16.x → 17.0) — Requires Lead Engineer approval, dedicated migration branch, Architecture Decision Record (ADR), and phased rollout.

2.4 Compatibility Matrix

Component	Required Version	Notes
Node.js	>= 20.0.0	LTS releases only. Node 22 preferred.
pnpm	>= 10.33	Workspace protocol, strict peer deps
Next.js	16.2.1	App Router primary, Pages Router legacy
React	19.2.4	Server Components, use hook, Actions
TypeScript	5.9.3	strict: true, noUncheckedIndexedAccess: true
Prisma	6.6.0	Client extensions, typed JSON
PostgreSQL	>= 15	Neon serverless (production)
SQLite	>= 3.40	better-sqlite3 (development)

2.5 Prohibited Dependencies

The following categories of packages must never be added:

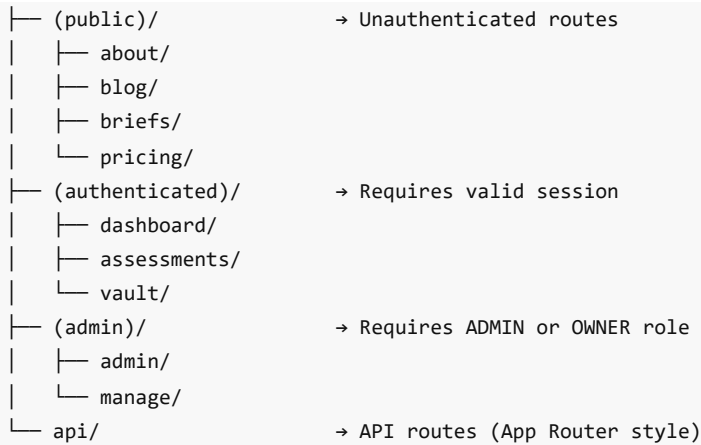
Category	Reason	Use Instead
jQuery	Incompatible with React model	Native DOM via refs
Moment.js	Enormous bundle, deprecated	date-fns or native Intl
Lodash (full)	Tree-shaking issues	Individual lodash-es/* imports or native
Express	Next.js handles routing	App Router API routes
Mongoose	We use PostgreSQL + Prisma	Prisma client
Firebase	Vendor lock-in, architectural conflict	Neon + Upstash

Chapter 3: Application Architecture

3.1 Next.js App Router

The App Router (`app/` directory) is the primary routing mechanism. All new routes must be created here.

app/	
├─ layout.tsx	→ Root layout (providers, global UI)
├─ page.tsx	→ Homepage
├─ loading.tsx	→ Global loading state
├─ error.tsx	→ Global error boundary
├─ not-found.tsx	→ 404 page



Route Groups — Parenthesised directories `(public)` , `(authenticated)` , `(admin)` create logical groupings without affecting the URL path. Each group can have its own `layout.tsx` that enforces access control.

Layouts — Layouts are Server Components that wrap child routes. They persist across navigation (no re-render). Use them for:

- Provider composition (root layout only)
- Navigation chrome (headers, sidebars)
- Auth enforcement (checking session in layout)

Loading States — Every route segment should have a `loading.tsx` that exports a skeleton or spinner. This enables streaming SSR with Suspense boundaries.

Error Boundaries — Every route segment should have an `error.tsx` that gracefully handles failures. Error boundaries must be Client Components (`'use client'`).

KEY PRINCIPLE

Routes are thin. A `page.tsx` file should contain minimal logic — it fetches data, passes it to a component, and returns JSX. Business logic lives in `Lib/` . UI logic lives in `components/` . Routes are the glue, nothing more.

3.2 Legacy Pages Router

The Pages Router (`pages/` directory) remains active for:

1. **NextAuth callbacks** — `pages/api/auth/[...nextauth].ts` (required by NextAuth 4.x)
2. **Debug endpoints** — `pages/api/debug/*` (development-only utilities)
3. **Legacy API routes** — Being migrated to App Router `app/api/`

⚠ WARNING

Do not create new routes in `pages/` . All new development uses the App Router. The Pages Router will be fully deprecated when NextAuth v5 migration is complete.

3.3 Rendering Strategies

Strategy	Use Case	Configuration
SSR (default)	Dynamic pages, personalised content, auth-gated	Default for all App Router pages

SSG	Static content pages, blog posts, briefs	<code>generateStaticParams()</code> + no dynamic data
ISR	Semi-static content that updates periodically	<code>revalidate: 3600</code> (1 hour default)
Edge	Low-latency API responses, middleware	<code>export const runtime = 'edge'</code>
Client	Interactive widgets, forms, real-time UI	<code>'use client'</code> directive

Default rule: Everything is SSR unless there is a specific reason to choose otherwise. SSR provides the best balance of performance, SEO, and data freshness.

ISR configuration:

```
// For content pages (blog, briefs, registry)
export const revalidate = 3600; // Re-generate every hour

// For highly dynamic pages
export const revalidate = 0; // Always fresh (equivalent to SSR)

// For truly static pages (legal, about)
export const dynamic = 'force-static';
```

3.4 Provider Composition

The root layout composes providers in a strict order. The order matters — inner providers may depend on outer providers being available.

```
// app/layout.tsx — Provider hierarchy (outermost → innermost)
<AuthProvider> // 1. Session context (NextAuth)
  <QueryProvider> // 2. Server state cache (React Query)
    <ThemeProvider> // 3. Light/dark mode
      <ZustandProvider> // 4. Client state stores
        <EmotionProvider> // 5. CSS-in-JS (legacy compat)
          {children}
        </EmotionProvider>
      </ZustandProvider>
    </ThemeProvider>
  </QueryProvider>
</AuthProvider>
```

⚠ WARNING

Do not add new providers without considering the render cost. Each provider wrapping `{children}` creates a potential re-render boundary. New providers require Lead Engineer approval and must be justified in terms of what they provide that cannot be achieved through existing patterns.

3.5 Server Components vs Client Components

Server Components (default — no directive needed):

- Access database directly via Prisma
- Read environment variables (server secrets)

- Use `async/await` at the component level
- Zero JavaScript sent to client
- Cannot use `useState`, `useEffect`, event handlers, or browser APIs

Client Components (`'use client'` directive at top of file):

- Interactive UI (forms, modals, toggles)
- Browser API access (localStorage, geolocation)
- Event handlers (onClick, onChange, onSubmit)
- React hooks (useState, useEffect, useRef)
- Third-party libraries that use browser APIs

Decision criteria — use a Client Component only when:

1. The component requires interactivity (user events)
2. The component uses React hooks that depend on client state
3. The component imports a library that accesses `window` or `document`

If none of these apply, it must remain a Server Component.

```
// ✓ CORRECT — Server Component (default)
// app/(public)/about/page.tsx
import { getFounderBio } from '@/lib/content';

export default async function AboutPage() {
  const bio = await getFounderBio();
  return <FounderProfile data={bio} />;
}

// ✓ CORRECT — Client Component (interactive)
// components/ui/ThemeToggle.tsx
'use client';
import { useState } from 'react';

export function ThemeToggle() {
  const [dark, setDark] = useState(false);
  return <button onClick={() => setDark(!dark)}>Toggle</button>;
}
```

KEY PRINCIPLE

The boundary between Server and Client Components is an architectural decision, not a convenience toggle. Push the `'use client'` boundary as deep into the component tree as possible. A page should be a Server Component that renders Client Component leaves — never the reverse.

3.6 Server Actions

Server Actions (`app/actions/`) are async functions that run on the server and can be called directly from Client Components without creating API endpoints.

```
// app/actions/assessment.ts
'use server';

import { z } from 'zod';
```

```
import { getAuthSession } from '@lib/auth';
import { prisma } from '@lib/prisma.server';

const SubmitSchema = z.object({
  assessmentId: z.string().uuid(),
  answers: z.array(z.object({
    questionId: z.string(),
    value: z.number().min(1).max(5),
  })),
});

export async function submitAssessment(formData: FormData) {
  const session = await getAuthSession();
  if (!session) throw new Error('Unauthorized');

  const parsed = SubmitSchema.parse(Object.fromEntries(formData));
  // ... process submission
}
```

Rules for Server Actions:

1. Always validate input with Zod before processing
2. Always check authentication/authorization
3. Never return sensitive data (internal IDs, secrets)
4. Use `revalidatePath()` or `revalidateTag()` after mutations
5. Throw errors for invalid states — the framework handles error UI

→ Cross-reference: Chapter 5 (Authentication) for guard patterns in Server Actions.

3.7 Middleware

The middleware (`middleware.ts`) runs at the Edge on every request before it reaches the route handler. It handles:

1. **Authentication enforcement** — Redirects unauthenticated users from protected routes
2. **Rate limiting** — Applies Upstash rate limits per IP/session
3. **Redirects** — Legacy URL redirects and canonical URL enforcement
4. **Headers** — Security headers (CSP, HSTS, X-Frame-Options)
5. **Geolocation** — Region-based routing (if applicable)

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { getToken } from 'next-auth/jwt';

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Protected routes require valid token
  if (pathname.startsWith('/dashboard') || pathname.startsWith('/vault')) {
    const token = await getToken({ req: request });
    if (!token) {
      return NextResponse.redirect(new URL('/auth/signin', request.url));
    }
  }
}
```

```

}

// Admin routes require role check
if (pathname.startsWith('/admin')) {
  const token = await getToken({ req: request });
  if (!token || ![ 'ADMIN', 'OWNER' ].includes(token.role as string)) {
    return NextResponse.redirect(new URL('/', request.url));
  }
}

return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*', '/vault/:path*', '/admin/:path*'],
};

```

⚠ WARNING

Middleware runs on every matched request at the Edge. It must be fast (< 50ms). Do not perform database queries, heavy computation, or external API calls in middleware. Use it for token checks and redirects only. Heavy authorization logic belongs in Server Components or Server Actions.

PART II — CORE SYSTEMS

Chapter 4: Database Layer

4.1 Prisma Architecture

Prisma 6.6.0 serves as the ORM and schema management layer. It provides:

- **Declarative schema** — `prisma/schema.prisma` is the single source of truth for all data models
- **Type-safe client** — Generated TypeScript types for every model, relation, and query
- **Migration engine** — Version-controlled schema changes
- **Query engine** — Optimized SQL generation with connection pooling

The Prisma client is instantiated as a singleton in `lib/prisma.server.ts` :

```

// lib/prisma.server.ts
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma =
  globalForPrisma.prisma ||
  new PrismaClient({
    log: process.env.NODE_ENV === 'development' ? ['query', 'warn', 'error'] : ['error'],
  });

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;

```

KEY PRINCIPLE

The singleton pattern prevents connection exhaustion during development (hot reloading creates new module instances). In production, Next.js standalone output ensures a single instance. Never instantiate `PrismaClient` anywhere other than this file.

4.2 Schema Overview — Domain Groupings

The schema is organized into six logical domains:

Core (Identity and Users)

Model	Purpose
User	Platform identity — email, name, role, hashedPassword, mfaSecret
Account	OAuth provider linkage (Google, GitHub)
Session	Active sessions (if using database strategy)
VerificationToken	Email verification, password reset

Enterprise (Client and CRM)

Model	Purpose
Organisation	Client company entity
Contact	Individual within an organisation
Engagement	Active service engagement (retainer, project)
Interaction	Logged touchpoints (calls, emails, meetings)
Pipeline	Sales pipeline stage tracking

Assessment (Decision Infrastructure)

Model	Purpose
Assessment	Assessment instance (linked to user)
AssessmentTemplate	Reusable assessment structure
Question	Individual assessment question
Response	User's answer to a question
Score	Computed assessment score
Report	Generated assessment report (PDF link, data)

Content (Publishing)

Model	Purpose
Post	Blog posts (mirrored from MDX for search/query)
Brief	Decision briefs

Resource	Downloadable resources
ContentView	View tracking for analytics

Governance (Access Control)

Model	Purpose
Role	Role definitions (USER, ADMIN, OWNER)
Permission	Granular permission definitions
AccessTier	Content tier membership
AuditLog	All privileged actions logged

Entitlements (Payments and Subscriptions)

Model	Purpose
Subscription	Stripe subscription linkage
Payment	Payment records
Invoice	Invoice generation records
Entitlement	Feature/content access grants

4.3 Key Enumerations

```
enum Role {
  USER          // Default – public access only
  ADMIN          // Platform administration
  OWNER          // Founder – unrestricted access
}

enum AccessTier {
  PUBLIC          // Available to all visitors
  MEMBER          // Requires free account
  INNER_CIRCLE    // Requires paid membership
  RESTRICTED      // Requires specific entitlement
  CLIENT          // Active paying client only
  ARCHITECT       // Senior client tier
  OWNER           // Founder-only content
  TOP_SECRET      // System internals, never exposed
}

enum AssessmentStatus {
  DRAFT           // Not yet started
  IN_PROGRESS     // Partially completed
  COMPLETED       // All questions answered
  SCORED          // Score computed
  REPORTED        // Report generated
}
```

```

}

enum EngagementStatus {
  PROSPECT      // Initial contact
  QUALIFYING    // In sales process
  ACTIVE        // Under contract
  PAUSED        // Temporarily suspended
  COMPLETED    // Engagement finished
  CHURNED       // Client lost
}

enum PipelineStage {
  AWARENESS     // Content consumption
  INTEREST      // Assessment/brief engagement
  EVALUATION    // Strategy Room trial
  COMMITMENT    // Proposal/contract
  RETENTION     // Active retainer
}

```

4.4 Connection Patterns

Development (SQLite):

```
DATABASE_URL="file:./dev.db"
```

SQLite provides zero-configuration local development. The schema is applied via `prisma db push` (no migration history needed locally).

Production (Neon PostgreSQL):

```
DATABASE_URL="postgresql://user:pass@ep-xxx.neon.tech/dbname?sslmode=require"
```

Neon provides serverless PostgreSQL with:

- Auto-scaling to zero (cost efficiency)
- Branching (preview environments get their own database branch)
- Connection pooling via PgBouncer (handles serverless connection storms)

⚠ WARNING

SQLite and PostgreSQL have syntax differences. Always test migrations against PostgreSQL before deploying. Key differences: no `ENUM` type in SQLite (uses `TEXT`), no `JSONB` (uses `TEXT`), different date handling. Prisma abstracts most of this, but raw SQL queries must be database-aware.

4.5 Migration Strategy

Environment	Command	Behaviour
Development	<code>pnpm prisma db push</code>	Applies schema changes directly, no migration file
Preview	<code>pnpm prisma migrate deploy</code>	Applies pending migrations from <code>prisma/migrations/</code>
Production	<code>pnpm prisma migrate deploy</code>	Applies pending migrations (CI/CD pipeline only)

Creating a new migration:

```
pnpm prisma migrate dev --name descriptive_name
```

This generates a timestamped SQL file in `prisma/migrations/` and applies it to the dev database.

Rules:

- 1. Never edit a migration file after it has been committed
- 2. Never delete a migration file — use a new migration to reverse changes
- 3. Migration names must be descriptive: `add_mfa_fields_to_user` , not `update_schema`
- 4. Destructive migrations (dropping columns/tables) require explicit `-- WARNING: DESTRUCTIVE` comment

4.6 Commands Reference

Command	Purpose	When to Use
<code>pnpm prisma generate</code>	Regenerate TypeScript client	After any schema change
<code>pnpm prisma db push</code>	Push schema to dev database	Local development iteration
<code>pnpm prisma migrate dev</code>	Create migration + apply	When schema change is final
<code>pnpm prisma migrate deploy</code>	Apply pending migrations	CI/CD production deploy
<code>pnpm prisma migrate reset</code>	Drop DB + reapply all migrations + seed	Nuclear reset (dev only)
<code>pnpm prisma studio</code>	Visual database browser	Debugging data issues
<code>pnpm prisma db seed</code>	Run seed script	Fresh database setup
<code>pnpm prisma format</code>	Format schema file	Before committing schema changes

4.7 Seeding

The seed script (`prisma/seed.ts`) populates the development database with representative data:

```
// prisma/seed.ts
import { PrismaClient } from '@prisma/client';
import { hash } from 'argon2';

const prisma = new PrismaClient();

async function main() {
  // Create admin user
  await prisma.user.upsert({
    where: { email: 'admin@abrahamoflondon.com' },
    update: {},
    create: {
      email: 'admin@abrahamoflondon.com',
      name: 'Admin',
      hashedPassword: await hash('dev-password-only'),
      role: 'OWNER',
    },
  });
}
```

```

    // Create sample assessments, content, etc.
    // ...
  }

  main()
    .catch(console.error)
    .finally(() => prisma.$disconnect());

```

Seeding is triggered by `pnpm prisma db seed` or automatically during `pnpm prisma migrate reset`.

4.8 Server-Only Import Rule

KEY PRINCIPLE

The Prisma client must never be imported in code that could execute in the browser. This includes Client Components, utility functions imported by Client Components, or any file without explicit server-only guarantees.

Enforcement mechanisms:

1. The singleton lives in `lib/prisma.server.ts` — the `.server.ts` suffix triggers Next.js to error if imported from a client bundle
2. ESLint rules flag `@prisma/client` imports outside approved paths
3. The build will fail if Prisma appears in client chunks

```

// ✓ CORRECT — Server Component
import { prisma } from '@lib/prisma.server';

export default async function UsersPage() {
  const users = await prisma.user.findMany();
  return <UserList users={users} />;
}

// ✗ PROHIBITED — Client Component importing Prisma
'use client';
import { prisma } from '@lib/prisma.server'; // BUILD ERROR

```

Chapter 5: Authentication and Authorization

5.1 NextAuth Configuration

Authentication is powered by NextAuth 4.24.13 with a JWT session strategy. The configuration lives in the Pages Router API route (required by NextAuth 4.x):

```
pages/api/auth/[...nextauth].ts
```

Core configuration:

```

export const authOptions: NextAuthOptions = {
  providers: [GoogleProvider, GitHubProvider, CredentialsProvider],
  adapter: PrismaAdapter(prisma),
  session: { strategy: 'jwt', maxAge: 30 * 24 * 60 * 60 }, // 30 days

```

```

jwt: { maxAge: 30 * 24 * 60 * 60 },
pages: {
  signIn: '/auth/signin',
  error: '/auth/error',
},
callbacks: { jwt, session, signIn },
};

```

5.2 OAuth Flow

Google OAuth:

- Scopes: openid , email , profile
- Used by: Public users, enterprise clients
- Auto-creates User + Account records on first sign-in

GitHub OAuth:

- Scopes: read:user , user:email
- Used by: Developers, internal team
- Links to existing account if email matches

Flow:

1. User clicks "Sign in with Google/GitHub"
2. Redirect to provider consent screen
3. Provider redirects back with authorization code
4. NextAuth exchanges code for tokens
5. User profile extracted → User upserted → Account linked
6. JWT issued with user ID, role, email
7. Redirect to callback URL or dashboard

5.3 Credentials Flow

The Credentials provider enables email/password authentication for admin users:

```

CredentialsProvider({
  name: 'credentials',
  credentials: {
    email: { type: 'email' },
    password: { type: 'password' },
    totp: { type: 'text' }, // MFA code (optional)
  },
  async authorize(credentials) {
    const user = await prisma.user.findUnique({
      where: { email: credentials.email },
    });
    if (!user?.hashedPassword) return null;

    const valid = await verify(user.hashedPassword, credentials.password);
    if (!valid) return null;

    // MFA check if enabled
    if (user.mfaSecret) {

```

```

    if (!credentials.totp) throw new Error('MFA_REQUIRED');
    const totpValid = authenticator.verify({
      token: credentials.totp,
      secret: user.mfaSecret,
    });
    if (!totpValid) throw new Error('MFA_INVALID');
  }

  return { id: user.id, email: user.email, role: user.role };
},
})

```

⚠ WARNING

The Credentials provider is restricted to admin/owner accounts. Public users must authenticate via OAuth. This prevents password-related support burden and ensures strong identity verification via trusted providers.

5.4 JWT Structure and Claims

```

interface JWTPayload {
  sub: string;      // User ID
  email: string;    // User email
  name: string;     // Display name
  role: Role;       // USER | ADMIN | OWNER
  tier: AccessTier;  // Highest access tier
  iat: number;      // Issued at
  exp: number;      // Expiry (30 days from issue)
}

```

The JWT is signed with `NEXTAUTH_SECRET` (HS256). It is stored in an HTTP-only cookie and refreshed on each request (sliding window).

5.5 Session Access Patterns

```

// Server Component / Server Action / API Route
import { getSession } from '@lib/auth';

export default async function ProtectedPage() {
  const session = await getSession();
  if (!session) redirect('/auth/signin');

  // session.user.id, session.user.role, session.user.email available
  return <Dashboard user={session.user} />;
}

// Client Component
'use client';
import { useSession } from 'next-auth/react';

export function UserMenu() {
  const { data: session, status } = useSession();
}

```

```

    if (status === 'loading') return <Skeleton />;
    if (!session) return <SignInButton />;
    return <Avatar user={session.user} />;
  }

```

5.6 Role System

Role	Capabilities	Assignment
USER	Public content, own assessments, own dashboard	Default on sign-up
ADMIN	All USER + user management, content admin, system config	Manual promotion
OWNER	All ADMIN + financial data, delete operations, system override	Bootstrap only

Roles are **hierarchical** — OWNER includes all ADMIN permissions, which includes all USER permissions.

5.7 Access Tier Hierarchy

Access tiers control content visibility independently of roles. A USER with an INNER_CIRCLE tier can see premium content but cannot administer the platform.

PUBLIC → MEMBER → INNER_CIRCLE → RESTRICTED → CLIENT → ARCHITECT → OWNER → TOP_SECRET

Each tier includes access to all content at its level and below. Tier assignment is managed through:

- Subscription status (Stripe webhook updates tier)
- Manual assignment (admin panel)
- Engagement status (active clients get CLIENT tier automatically)

5.8 Guard Functions

```

// lib/auth/guards.ts

export async function requireAuth() {
  const session = await getAuthSession();
  if (!session) throw new AuthError('UNAUTHENTICATED');
  return session;
}

export async function requireRole(minimumRole: Role) {
  const session = await requireAuth();
  const hierarchy = { USER: 0, ADMIN: 1, OWNER: 2 };
  if (hierarchy[session.user.role] < hierarchy[minimumRole]) {
    throw new AuthError('INSUFFICIENT_ROLE');
  }
  return session;
}

export async function requireTier(minimumTier: AccessTier) {
  const session = await requireAuth();
  const tierOrder = [
    'PUBLIC', 'MEMBER', 'INNER_CIRCLE', 'RESTRICTED',

```

```

    'CLIENT', 'ARCHITECT', 'OWNER', 'TOP_SECRET'
  ];
  if (tierOrder.indexOf(session.user.tier) < tierOrder.indexOf(minimumTier)) {
    throw new AuthError('INSUFFICIENT_TIER');
  }
  return session;
}

```

Usage in Server Actions:

```

'use server';
export async function deleteUser(userId: string) {
  const session = await requireRole('OWNER'); // Only founder can delete users
  await prisma.user.delete({ where: { id: userId } });
  await logAuditEvent('USER_DELETED', session.user.id, { targetUser: userId });
}

```

5.9 Cookie Configuration

```

cookies: {
  sessionToken: {
    name: '__Secure-next-auth.session-token',
    options: {
      httpOnly: true,      // No JavaScript access
      sameSite: 'lax',     // CSRF protection
      path: '/',
      secure: true,        // HTTPS only (production)
      domain: '.abrahamoflondon.com',
    },
  },
},
}

```

KEY PRINCIPLE

Authentication cookies are always `httpOnly` and `secure` in production. The `__Secure-` prefix is a browser-enforced signal that the cookie was set over HTTPS. Never weaken these settings — they are the primary defence against session hijacking.

5.10 Multi-Factor Authentication

MFA uses TOTP (Time-based One-Time Password) via the `otplib` library:

1. **Enrollment:** User generates secret → stored encrypted in `User.mfaSecret`
2. **QR Code:** Secret encoded as `otpauth://` URI → rendered as QR for authenticator app
3. **Verification:** On login, user provides 6-digit code → verified against secret with 30s window
4. **Recovery:** Backup codes generated at enrollment → hashed and stored

MFA is **required** for ADMIN and OWNER roles. USER role may optionally enable it.

5.11 Admin Bootstrap

First-time deployment uses the `BOOTSTRAP_ADMIN_EMAILS` environment variable:


```
BOOTSTRAP_ADMIN_EMAILS="founder@abrahamoflondon.com"
```

When a user with a matching email signs in for the first time, they are automatically assigned the OWNER role. This eliminates the need for direct database manipulation during initial setup.

⚠ WARNING

Remove or clear `BOOTSTRAP_ADMIN_EMAILS` after initial setup is complete. Leaving it active means anyone who gains access to an email address on that list can claim owner privileges on their first sign-in.

Chapter 6: Content System

6.1 Contentlayer2 Architecture

Contentlayer2 transforms MDX files in the `content/` directory into type-safe JSON data accessible at build time. It operates as a build-time content processing pipeline:

```
content/*.mdx → Contentlayer2 → .contentlayer/generated/ → import in components
```

Configuration lives in `contentlayer.config.ts` at the project root. The generated output provides:

- Typed document objects with computed fields
- Collection arrays for listing pages
- Individual document lookup by slug

```
// Importing generated content
import { allBriefs } from 'contentlayer/generated';
// or via compatibility layer for Windows:
import { allBriefs } from '@lib/contentlayer-generated';
```

6.2 Document Type Definitions

The platform defines 20+ document types. Key types include:

```
// contentlayer.config.ts (simplified)
export const Brief = defineDocumentType(() => ({
  name: 'Brief',
  filePathPattern: 'briefs/**/*.mdx',
  contentType: 'mdx',
  fields: {
    title: { type: 'string', required: true },
    description: { type: 'string', required: true },
    date: { type: 'date', required: true },
    author: { type: 'string', required: true },
    category: { type: 'string', required: true },
    tier: { type: 'enum', options: ['public', 'member', 'inner_circle', 'client'], required: true },
    tags: { type: 'list', of: { type: 'string' } },
    featured: { type: 'boolean', default: false },
    draft: { type: 'boolean', default: false },
    image: { type: 'string' },
  },
});
```

```

    readingTime: { type: 'string' }, // Computed
  },
  computedFields: {
    slug: { type: 'string', resolve: (doc) => doc._raw.flattenedPath.replace('briefs/', '') },
  },
  url: { type: 'string', resolve: (doc) => `/briefs/${doc.slug}` },
  readingTime: { type: 'string', resolve: (doc) => calculateReadingTime(doc.body.raw) },
},
}));

export const BlogPost = defineDocumentType(() => ({
  name: 'BlogPost',
  filePathPattern: 'blog/**/*.mdx',
  contentType: 'mdx',
  fields: {
    title: { type: 'string', required: true },
    description: { type: 'string', required: true },
    date: { type: 'date', required: true },
    author: { type: 'string', required: true },
    category: { type: 'string', required: true },
    tags: { type: 'list', of: { type: 'string' } },
    featured: { type: 'boolean', default: false },
    draft: { type: 'boolean', default: false },
    image: { type: 'string' },
    canonical: { type: 'string' },
  },
  // computedFields similar to Brief
}));

```

Additional document types: VaultEntry , RegistryItem , CaseStudy , Framework , Lexicon , Toolkit , Playbook , Template , Assessment , Testimonial , and others.

6.3 Frontmatter Schema

Every MDX file must include valid frontmatter. The following fields are universal:

Field	Type	Required	Description
title	string	Yes	Display title (max 70 chars for SEO)
description	string	Yes	Meta description (max 160 chars)
date	ISO date	Yes	Publication date
author	string	Yes	Author identifier
draft	boolean	No	If true, excluded from production builds

Type-specific required fields:

Document Type	Additional Required Fields
Brief	category, tier

BlogPost	category
VaultEntry	tier, access_requirement
CaseStudy	client_type, outcome, duration
Framework	version, domain
Lexicon	term, definition_short

⚠ WARNING

Missing required frontmatter fields cause build failures. Contentlayer2 validates at build time and will reject documents with schema violations. Always run `pnpm contentlayer build` locally before committing new content.

6.4 Processing Pipeline

Content flows through a six-stage pipeline:

1. PARSE → Read MDX file, extract frontmatter (gray-matter)
2. VALIDATE → Check frontmatter against document type schema
3. COMPUTE → Generate computed fields (slug, URL, reading time, TOC)
4. TRANSFORM → Apply remark/rehype plugins to MDX body
5. SANITIZE → Strip potentially dangerous HTML (DOMPurify)
6. OUTPUT → Write typed JSON to `.contentlayer/generated/`

Each stage is idempotent. A failure at any stage prevents the document from being included in the generated output but does not block other documents (unless `strict: true` is set in config).

6.5 Remark Plugins

Plugin	Purpose
remark-gfm	GitHub Flavored Markdown (tables, strikethrough, task lists, autolinks)
remark-reading-time	Computes estimated reading time from word count
remark-toc	Generates table of contents from headings
remark-unwrap-images	Removes wrapping <code><p></code> tags from images
remark-breaks	Converts single newlines to <code>
</code> (matches writing style)

6.6 Rehype Plugins

Plugin	Purpose
rehype-slug	Adds <code>id</code> attributes to headings (enables anchor links)
rehype-autolink-headings	Wraps heading text in clickable anchor
rehype-pretty-code	Syntax highlighting via Shiki (supports all languages)

Plugin order matters — `rehype-slug` must run before `rehype-autolink-headings` .

6.7 Generated Types Usage

Contentlayer2 generates TypeScript types for every document type:

```
// Auto-generated – do not edit
interface Brief {
  title: string;
  description: string;
  date: string;
  author: string;
  category: string;
  tier: 'public' | 'member' | 'inner_circle' | 'client';
  tags?: string[];
  featured: boolean;
  draft: boolean;
  image?: string;
  slug: string;
  url: string;
  readingTime: string;
  body: { raw: string; code: string };
  _id: string;
  _raw: RawDocumentData;
}
```

Usage pattern:

```
import { allBriefs, type Brief } from 'contentlayer/generated';

// List all published briefs
export function getPublishedBriefs(): Brief[] {
  return allBriefs
    .filter((brief) => !brief.draft)
    .sort((a, b) => new Date(b.date).getTime() - new Date(a.date).getTime());
}

// Get single brief by slug
export function getBriefBySlug(slug: string): Brief | undefined {
  return allBriefs.find((brief) => brief.slug === slug);
}
```

6.8 Content Routing

Content Type	Source Directory	Public Route	Access Control
Briefs	content/briefs/	/briefs/[slug]	Tier-based (per document)
Blog	content/blog/	/blog/[slug]	Public
Vault	content/vault/	/vault/[slug]	INNER_CIRCLE minimum
Registry	content/registry/	/registry/[slug]	MEMBER minimum

Lexicon	content/lexicon/	/lexicon/[slug]	Public
Frameworks	content/frameworks/	/frameworks/[slug]	Tier-based

Content pages use `generateStaticParams()` for static generation at build time with ISR for updates:

```
// app/(public)/briefs/[slug]/page.tsx
import { allBriefs } from 'contentlayer/generated';

export async function generateStaticParams() {
  return allBriefs
    .filter((brief) => !brief.draft)
    .map((brief) => ({ slug: brief.slug }));
}

export const revalidate = 3600; // ISR: regenerate hourly
```

6.9 ISR and Cache Strategy

Content Type	Revalidation Period	Rationale
Blog posts	3600s (1 hour)	Updated occasionally, freshness not critical
Briefs	3600s (1 hour)	Same as blog
Vault content	3600s (1 hour)	Gated content, low update frequency
Homepage	1800s (30 min)	Features rotating content
Pricing page	86400s (24 hours)	Rarely changes
Legal pages	force-static	Effectively never changes

KEY PRINCIPLE

ISR is the default caching strategy for content pages. It provides the performance of static generation with the freshness of server rendering. The 3600s default balances CDN cache efficiency against content update latency. Override only with justification.

6.10 Windows Compatibility

Contentlayer2 has known issues on Windows due to file path handling. The platform includes several compatibility layers:

1. `contentlayer.windows-fix.mjs` — Patches path resolution to use forward slashes
2. `contentlayer.windows.config.ts` — Windows-specific configuration overrides
3. `scripts/contentlayer-windows-fix.js` — Pre-build script that normalizes paths
4. `lib/contentlayer-generated.ts` — Compatibility import wrapper

```
// lib/contentlayer-generated.ts – Use this import path on Windows
// This wrapper handles the path resolution differences between
// Unix and Windows environments
export * from 'contentlayer/generated';
```

Development on Windows:

```
# Always run the Windows fix before building content
pnpm contentlayer:fix # Runs scripts/contentlayer-windows-fix.js
pnpm contentlayer build # Then build normally
```

⚠ WARNING

CI/CD runs on Linux. If content builds succeed locally on Windows but fail in CI, the issue is almost always path separators (\ vs /). Always use the compatibility wrapper for imports and test content builds in CI before merging.

End of Part A — Engineering Manual continues in `eng_manual_b.md` (Parts III–VII and Appendices).

Abraham of London — Engineering Manual (Part B)

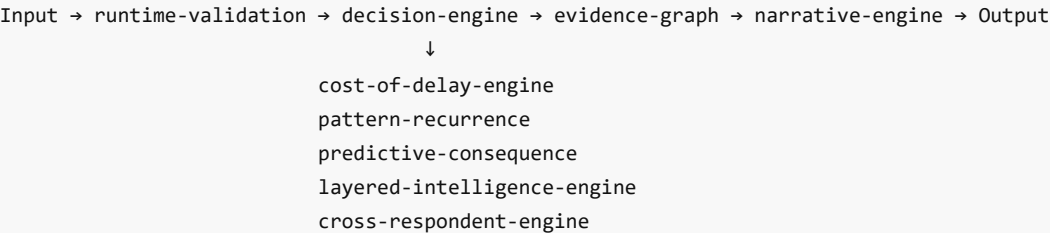
—— **KEY PRINCIPLE** This document is the authoritative reference for domain systems, presentation layer, API architecture, and PDF pipeline. Every engineer must read, understand, and follow these standards without exception.

PART III — DOMAIN SYSTEMS

Chapter 7: Diagnostic Engine

7.1 Architecture Overview

The diagnostic engine is the analytical core of the platform — over 50 source files in `lib/diagnostics/`. Data flows through a strict pipeline:



Every assessment begins with validated input, passes through scoring and evidence linking, and terminates in a natural-language narrative suitable for PDF rendering.

—— **KEY PRINCIPLE** The diagnostic pipeline is pure — each module receives typed input and produces typed output. No module may read from the database directly. All data access happens at the boundary (API route handlers) and is passed in as arguments.

7.2 Core Modules

Module	Path	Purpose
decision-engine.ts	lib/diagnostics/decision-engine.ts	Scores raw responses against weighted criteria, producing numeric severity and categorical

		bands
runtime-validation.ts	lib/diagnostics/runtime-validation.ts	Validates all input against Zod schemas before any processing begins — rejects malformed payloads
cross-respondent-engine.ts	lib/diagnostics/cross-respondent-engine.ts	Merges and reconciles responses from multiple respondents into a unified assessment
evidence-graph.ts	lib/diagnostics/evidence-graph.ts	Links scored indicators to source evidence, building a directed graph of causal relationships
cost-of-delay-engine.ts	lib/diagnostics/cost-of-delay-engine.ts	Calculates ROI impact of inaction — quantifies what happens if the client does nothing
layered-intelligence-engine.ts	lib/diagnostics/layered-intelligence-engine.ts	Multi-level analysis combining surface indicators, structural patterns, and deep signals
narrative-engine.ts	lib/diagnostics/narrative-engine.ts	Transforms structured analytical output into natural-language prose suitable for reports
pattern-recurrence.ts	lib/diagnostics/pattern-recurrence.ts	Detects recurring behavioural and structural patterns across time and context
predictive-consequence.ts	lib/diagnostics/predictive-consequence.ts	Projects future states based on current trajectory — models probable outcomes

7.3 Assessment Contracts and Result Builders

Every diagnostic module adheres to a typed contract:

```
// Input contract
interface DiagnosticInput {
  sessionId: string;
  respondentId: string;
  responses: ValidatedResponse[];
  context: AssessmentContext;
  options?: DiagnosticOptions;
}

// Output contract — all modules produce this shape
interface DiagnosticResult {
  scores: ScoredDimension[];
  evidence: EvidenceNode[];
  severity: SeverityLevel;
  confidence: number; // 0-1
  narrative?: string;
  metadata: ResultMetadata;
}
```

Result builders construct the final output:

```
// lib/diagnostics/builders/result-builder.ts
export function buildDiagnosticResult(
  scores: ScoredDimension[],
  evidence: EvidenceNode[],
  options: BuilderOptions
): DiagnosticResult {
  return {
    scores,
    evidence,
    severity: deriveSeverity(scores),
    confidence: calculateConfidence(evidence),
    narrative: options.includeNarrative
      ? generateNarrative(scores, evidence)
      : undefined,
    metadata: {
      generatedAt: new Date().toISOString(),
      engineVersion: ENGINE_VERSION,
      modulesUsed: options.modules,
    },
  };
}
```

⚠ WARNING Never construct a `DiagnosticResult` manually. Always use the result builder — it enforces invariants (e.g., severity must match score thresholds, confidence must reflect evidence density).

7.4 Database Models

```
model DiagnosticSession {
  id          String  @id @default(cuid())
  userId      String
  type        DiagnosticType
  status       SessionStatus // DRAFT, IN_PROGRESS, COMPLETED, EXPIRED
  startedAt   DateTime @default(now())
  completedAt  DateTime?
  expiresAt   DateTime
  responses    DiagnosticResponse[]
  reports      DiagnosticReport[]
  artifacts    DiagnosticArtifact[]
  user         User      @relation(fields: [userId], references: [id])

  @@index([userId, status])
  @@index([expiresAt])
}

model DiagnosticResponse {
  id          String  @id @default(cuid())
  sessionId   String
  questionId  String
  value       Json
  respondentId String?
```



```

    answeredAt DateTime @default(now())
    session      DiagnosticSession @relation(fields: [sessionId], references: [id])

    @@unique([sessionId, questionId, respondentId])
}

model DiagnosticReport {
  id          String  @id @default(cuid())
  sessionId   String
  status       ReportStatus // GENERATING, READY, FAILED, ARCHIVED
  format       ReportFormat // PDF, JSON, HTML
  url          String?
  data         Json?
  generatedAt DateTime?
  session      DiagnosticSession @relation(fields: [sessionId], references: [id])

  @@index([sessionId, status])
}

model DiagnosticArtifact {
  id          String  @id @default(cuid())
  sessionId   String
  type         ArtifactType // CHART, TABLE, EVIDENCE_MAP, EXECUTIVE_SUMMARY
  label        String
  data         Json
  order        Int      @default(0)
  session      DiagnosticSession @relation(fields: [sessionId], references: [id])
}

```

7.5 Enums

```

// Severity levels – ordered by urgency
enum SeverityLevel {
  NEGLIGIBLE = 'NEGLIGIBLE',
  LOW = 'LOW',
  MODERATE = 'MODERATE',
  HIGH = 'HIGH',
  CRITICAL = 'CRITICAL',
  SYSTEMIC = 'SYSTEMIC',
}

// Session lifecycle
enum SessionStatus {
  DRAFT = 'DRAFT',
  IN_PROGRESS = 'IN_PROGRESS',
  COMPLETED = 'COMPLETED',
  EXPIRED = 'EXPIRED',
}

// Report status

```

```
enum ReportStatus {
  GENERATING = 'GENERATING',
  READY = 'READY',
  FAILED = 'FAILED',
  ARCHIVED = 'ARCHIVED',
}
```

7.6 API Routes for Diagnostics

Method	Route	Purpose
POST	/api/diagnostics/sessions	Create a new diagnostic session
GET	/api/diagnostics/sessions/[id]	Retrieve session with responses
PATCH	/api/diagnostics/sessions/[id]	Update session status
POST	/api/diagnostics/sessions/[id]/responses	Submit responses
POST	/api/diagnostics/sessions/[id]/generate	Trigger report generation
GET	/api/diagnostics/sessions/[id]/report	Retrieve generated report
GET	/api/diagnostics/sessions/[id]/artifacts	List artifacts for session
DELETE	/api/diagnostics/sessions/[id]	Soft-delete session

All routes follow the standard response format:

```
// Success
{ ok: true, data: { ... }, code: 200 }

// Error
{ ok: false, error: "Descriptive message", code: 422 }
```

7.7 Storage

Diagnostic artifacts and generated PDFs use a configurable storage backend:

```
// lib/diagnostics/storage.ts
interface StorageAdapter {
  put(key: string, data: Buffer, options?: StorageOptions): Promise<string>;
  get(key: string): Promise<Buffer | null>;
  delete(key: string): Promise<void>;
  getSignedUrl(key: string, expiresIn?: number): Promise<string>;
}

// Environment-driven selection
const storage = process.env.STORAGE_BACKEND === 's3'
  ? new S3StorageAdapter({
      bucket: process.env.S3_BUCKET!,
      region: process.env.S3_REGION!,
    })
```

```
    })
  : new LocalStorageAdapter({
    basePath: path.join(process.cwd(), '.storage/diagnostics'),
  });
```

———— **KEY PRINCIPLE** Local storage is for development only. Production **MUST** use S3. The adapter interface ensures code never cares which backend is active — all access goes through the adapter.

7.8 PDF Generation from Diagnostics

Dagnostic reports generate PDFs through the unified pipeline (see Chapter 17). The diagnostic-specific flow:

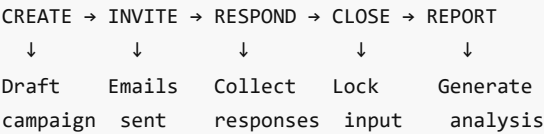
DagnosticResult → template selection → data hydration → React PDF render → storage

Templates are selected based on DiagnosticType and ReportFormat :

```
// lib/diagnostics/pdf/template-selector.ts
export function selectTemplate(
  type: DiagnosticType,
  format: ReportFormat
): PDFTemplate {
  const key = `${type}:${format}`;
  const template = TEMPLATE_REGISTRY.get(key);
  if (!template) {
    throw new DiagnosticError(`No template registered for ${key}`);
  }
  return template;
}
```

Chapter 8: Alignment & Campaign System

8.1 Campaign Lifecycle



Each transition is gated:

Transition	Gate
CREATE → INVITE	Campaign must have at least one participant added
INVITE → RESPOND	At least one invite accepted (tracked via CampaignParticipant.status)
RESPOND → CLOSE	Owner explicitly closes OR deadline reached
CLOSE → REPORT	All responses validated, cross-respondent merge complete

⚠ WARNING Once a campaign moves to CLOSE, no further responses are accepted. The `cross-respondent-engine` runs on the frozen dataset. This is irreversible.

8.2 Organization Model

```
model Organisation {
  id          String  @id @default(cuid())
  name        String
  slug        String  @unique
  tier         OrgTier // FREE, PROFESSIONAL, ENTERPRISE
  logoUrl     String?
  domain      String? @unique
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  memberships OrganisationMembership[]
  campaigns   AlignmentCampaign[]

  @@index([slug])
  @@index([domain])
}

model OrganisationMembership {
  id          String  @id @default(cuid())
  organisationId String
  userId      String
  role        OrgRole // OWNER, ADMIN, MEMBER, VIEWER
  joinedAt    DateTime @default(now())
  organisation Organisation @relation(fields: [organisationId], references: [id])
  user        User        @relation(fields: [userId], references: [id])

  @@unique([organisationId, userId])
}

enum OrgTier {
  FREE
  PROFESSIONAL
  ENTERPRISE
}

enum OrgRole {
  OWNER
  ADMIN
  MEMBER
  VIEWER
}
```

8.3 AlignmentCampaign and CampaignParticipant

```

model AlignmentCampaign {
  id          String   @id @default(cuid())
  organisationId String
  title       String
  description  String?
  status      CampaignStatus // DRAFT, ACTIVE, CLOSED, ARCHIVED
  domains     Json        // Array of scoring domains
  deadline    DateTime?
  createdById String
  createdAt   DateTime @default(now())
  closedAt    DateTime?
  organisation Organisation @relation(fields: [organisationId], references: [id])
  participants CampaignParticipant[]

  @@index([organisationId, status])
}

model CampaignParticipant {
  id          String   @id @default(cuid())
  campaignId  String
  email       String
  userId      String?
  role        ParticipantRole // SUBJECT, RESPONDENT, OBSERVER
  status      ParticipantStatus // INVITED, ACCEPTED, RESPONDED, DECLINED
  invitedAt   DateTime @default(now())
  respondedAt DateTime?
  campaign    AlignmentCampaign @relation(fields: [campaignId], references: [id])

  @@unique([campaignId, email])
}

```

8.4 Scoring Domains

The alignment system evaluates across six fundamental domains:

Domain	Code	Description
Identity	IDENTITY	Clarity of self-concept, values articulation, role congruence
Decision	DECISION	Decision-making quality, speed, consistency with stated values
Environment	ENVIRONMENT	Physical and relational context, structural enablers/blockers
Behaviour	BEHAVIOUR	Observable actions vs. stated intentions, habit architecture
Emotional Order	EMOTIONAL_ORDER	Emotional regulation, response patterns, resilience indicators
Legacy	LEGACY	Long-term orientation, generational thinking, impact trajectory

Each domain produces a score from 0–100 and maps to an `AlignmentBand` .

8.5 AlignmentBand Classification

```
enum AlignmentBand {
  ALIGNED = 'ALIGNED',          // 75-100 – coherent, functional, generative
  DRIFTING = 'DRIFTING',        // 50-74  – losing coherence, early intervention warranted
  MISALIGNED = 'MISALIGNED',    // 25-49  – structural contradiction, active harm possible
  DISORDERED = 'DISORDERED',    // 0-24   – systemic breakdown, urgent intervention required
}

// Classification logic
export function classifyBand(score: number): AlignmentBand {
  if (score >= 75) return AlignmentBand.ALIGNED;
  if (score >= 50) return AlignmentBand.DRIFTING;
  if (score >= 25) return AlignmentBand.MISALIGNED;
  return AlignmentBand.DISORDERED;
}
```

—— **KEY PRINCIPLE** Band thresholds are institutional constants. They do not change per client, per campaign, or per domain. The scoring algorithm may weight domains differently, but the band boundaries are absolute.

8.6 Enterprise Assessment Generation

Enterprise assessments aggregate across multiple campaigns and participants:

```
// lib/alignment/enterprise/assessment-generator.ts
export async function generateEnterpriseAssessment(
  organisationId: string,
  options: EnterpriseAssessmentOptions
): Promise<EnterpriseAssessment> {
  const campaigns = await getCampaignsForOrg(organisationId, {
    status: 'CLOSED',
    dateRange: options.dateRange,
  });

  const aggregated = aggregateScores(campaigns);
  const interventions = deriveInterventions(aggregated);
  const narrative = await generateEnterpriseNarrative(aggregated, interventions);

  return {
    organisationId,
    generatedAt: new Date().toISOString(),
    domainScores: aggregated.domainScores,
    overallBand: classifyBand(aggregated.overallScore),
    interventions,
    narrative,
    participantCount: aggregated.totalParticipants,
    campaignCount: campaigns.length,
  };
}
```

8.7 Strategic Interventions

Interventions are generated based on band classification and domain-specific rules:

```
interface StrategicIntervention {
  id: string;
  domain: ScoringDomain;
  severity: SeverityLevel;
  title: string;
  description: string;
  recommendedActions: string[];
  timeframe: 'IMMEDIATE' | '30_DAYS' | '90_DAYS' | '6_MONTHS';
  estimatedImpact: number; // 0-1
}
```

Interventions follow a priority matrix:

Band	Timeframe	Action Type
DISORDERED	IMMEDIATE	Crisis stabilisation
MISALIGNED	30_DAYS	Structural correction
DRIFTING	90_DAYS	Course adjustment
ALIGNED	6_MONTHS	Optimisation / maintenance

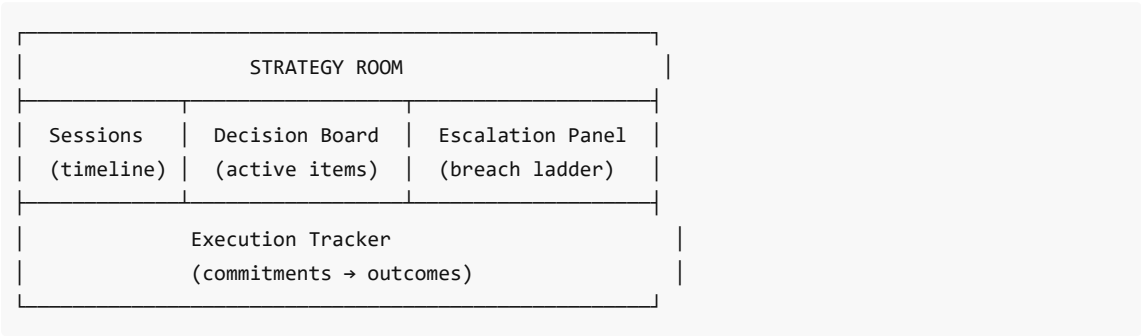
8.8 API Route Reference

Method	Route	Purpose
POST	/api/alignment/campaigns	Create campaign
GET	/api/alignment/campaigns	List campaigns for org
GET	/api/alignment/campaigns/[id]	Get campaign detail
PATCH	/api/alignment/campaigns/[id]	Update campaign (status, metadata)
POST	/api/alignment/campaigns/[id]/invite	Invite participants
POST	/api/alignment/campaigns/[id]/respond	Submit alignment response
POST	/api/alignment/campaigns/[id]/close	Close campaign
GET	/api/alignment/campaigns/[id]/report	Get alignment report
POST	/api/alignment/enterprise/assess	Generate enterprise assessment
GET	/api/alignment/organisations/[id]	Get organisation detail
POST	/api/alignment/organisations	Create organisation
PATCH	/api/alignment/organisations/[id]/members	Manage membership

Chapter 9: Strategy Room & Execution

9.1 Strategy Room Architecture

The Strategy Room is a real-time workspace for decision-making and strategic planning:



Core components:

- **Session Manager** — creates, archives, and retrieves strategy sessions
- **Decision Board** — tracks decisions from proposal through execution
- **Breach Ladder** — escalation framework for unresolved commitments
- **Execution Tracker** — maps commitments to measurable outcomes

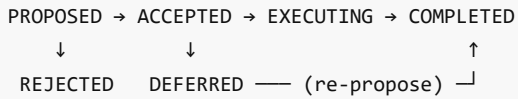
9.2 Session Management

```
interface StrategySession {
  id: string;
  userId: string;
  title: string;
  status: 'ACTIVE' | 'PAUSED' | 'COMPLETED' | 'ABANDONED';
  decisions: Decision[];
  commitments: Commitment[];
  createdAt: string;
  lastActivityAt: string;
  completedAt?: string;
}

interface Decision {
  id: string;
  sessionId: string;
  statement: string;
  status: 'PROPOSED' | 'ACCEPTED' | 'REJECTED' | 'DEFERRED';
  rationale?: string;
  deadline?: string;
  owner: string;
  createdAt: string;
}
```

9.3 Decision Tracking

Decisions flow through a state machine:



Every decision must have:

1. A clear statement (what is being decided)
2. An owner (who is accountable)
3. A deadline (when it must be resolved)
4. A rationale (why this decision, not another)

⚠ WARNING Decisions without deadlines are not decisions — they are wishes. The system enforces deadline assignment at creation time. A decision with `deadLine: null` cannot transition to `ACCEPTED`.

9.4 Breach Ladder Implementation

The breach ladder is a time-based escalation framework for commitments that are not being honoured:

```
interface BreachLadder {
  commitmentId: string;
  currentLevel: BreachLevel;
  history: BreachEvent[];
  nextEscalation: string; // ISO datetime
}

enum BreachLevel {
  NONE = 0, // Commitment on track
  NOTICE = 1, // 24h past deadline – gentle reminder
  WARNING = 2, // 48h – formal warning
  ESCALATION = 3, // 72h – escalated to stakeholders
  BREACH = 4, // 5 days – formal breach recorded
  CRITICAL = 5, // 7 days – systemic failure flag
}
```

9.5 Escalation Engine (24h → 48h → 72h → 5d → 7d)

The escalation engine runs on a cron schedule and processes all active commitments:

```
// lib/strategy/escalation-engine.ts
export async function processEscalations(): Promise<EscalationResult[]> {
  const overdue = await getOverdueCommitments();
  const results: EscalationResult[] = [];

  for (const commitment of overdue) {
    const hoursOverdue = differenceInHours(new Date(), commitment.deadline);
    const newLevel = determineBreachLevel(hoursOverdue);

    if (newLevel > commitment.breachLevel) {
      await escalate(commitment, newLevel);
    }
  }
}
```

```

    results.push({
      commitmentId: commitment.id,
      previousLevel: commitment.breachLevel,
      newLevel,
      action: getEscalationAction(newLevel),
    });
  }
}

return results;
}

function determineBreachLevel(hoursOverdue: number): BreachLevel {
  if (hoursOverdue >= 168) return BreachLevel.CRITICAL; // 7 days
  if (hoursOverdue >= 120) return BreachLevel.BREACH; // 5 days
  if (hoursOverdue >= 72) return BreachLevel.ESCALATION;
  if (hoursOverdue >= 48) return BreachLevel.WARNING;
  if (hoursOverdue >= 24) return BreachLevel.NOTICE;
  return BreachLevel.NONE;
}

```

Escalation actions by level:

Level	Hours Overdue	Action
NOTICE	24h	In-app notification to commitment owner
WARNING	48h	Email to owner + session log entry
ESCALATION	72h	Notification to all session stakeholders
BREACH	5 days	Formal breach record, visible in reports
CRITICAL	7 days	Systemic failure flag, blocks new commitments

—— **KEY PRINCIPLE** *The escalation engine is mechanical and impartial. It does not negotiate, it does not accept excuses, it does not reset without explicit action. A commitment is either honoured or it is breached — there is no middle ground.*

Chapter 10: Commercial Layer

10.1 Stripe Integration Patterns

All payment processing flows through Stripe. The integration follows a strict pattern:

```

// lib/stripe/client.ts
import Stripe from 'stripe';

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2024-04-10',
  typescript: true,
});

```

```
// Webhook signature verification – ALWAYS verify
export function verifyWebhookSignature(
  payload: string | Buffer,
  signature: string
): Stripe.Event {
  return stripe.webhooks.constructEvent(
    payload,
    signature,
    process.env.STRIPE_WEBHOOK_SECRET!
  );
}
```

⚠ WARNING Never trust client-side payment confirmations. All payment state changes must be driven by verified Stripe webhooks. The client may display optimistic UI, but entitlements are only granted after webhook confirmation.

10.2 Pricing Engine

```
// lib/pricing/engine.ts
interface PricingTier {
  id: string;
  name: string;
  stripePriceId: string;
  features: Feature[];
  limits: TierLimits;
  amount: number; // in smallest currency unit (pence)
  currency: string; // ISO 4217
  interval: 'month' | 'year' | 'once';
}

interface TierLimits {
  diagnosticSessions: number; // -1 = unlimited
  campaignParticipants: number;
  storageBytes: number;
  pdfGenerations: number;
  apiCalls: number;
}

export function getPricingForTier(tierId: string): PricingTier {
  const tier = PRICING_REGISTRY.get(tierId);
  if (!tier) throw new PricingError(`Unknown tier: ${tierId}`);
  return tier;
}
```

10.3 Checkout Flow

Client → /api/checkout/create-session → Stripe Checkout → Webhook → Entitlement

API routes:

Method	Route	Purpose
POST	/api/checkout/create-session	Creates Stripe Checkout session
POST	/api/checkout/webhook	Handles Stripe webhook events
GET	/api/checkout/portal	Creates Stripe Customer Portal session
GET	/api/checkout/status/[sessionId]	Check checkout session status

```
// app/api/checkout/create-session/route.ts
export async function POST(req: Request) {
  const session = await getAuthSession();
  if (!session) return unauthorized();

  const body = await req.json();
  const { priceId, successUrl, cancelUrl } = checkoutSchema.parse(body);

  const checkoutSession = await stripe.checkout.sessions.create({
    customer: await getOrCreateCustomer(session.user.id),
    line_items: [{ price: priceId, quantity: 1 }],
    mode: determineModeFromPrice(priceId),
    success_url: successUrl,
    cancel_url: cancelUrl,
    metadata: { userId: session.user.id },
  });

  return json({ ok: true, data: { url: checkoutSession.url }, code: 200 });
}
```

10.4 Entitlement System

Entitlements grant access to features and resources:

```
enum EntitlementType {
  TIER = 'TIER',           // Subscription-based access level
  PRODUCT = 'PRODUCT',    // One-time purchase (e.g., a specific report)
  ARTIFACT = 'ARTIFACT',  // Access to a specific generated artifact
}

interface Entitlement {
  id: string;
  userId: string;
  type: EntitlementType;
  reference: string;       // Stripe subscription/payment ID
  grantedAt: string;
  expiresAt?: string;     // null = permanent
  revoked: boolean;
  metadata?: Record<string, unknown>;
}
```

```

}

// Check entitlement
export async function hasEntitlement(
  userId: string,
  type: EntitlementType,
  reference?: string
): Promise<boolean> {
  const entitlement = await db.entitlement.findFirst({
    where: {
      userId,
      type,
      ...(reference ? { reference } : {}),
      revoked: false,
      OR: [
        { expiresAt: null },
        { expiresAt: { gt: new Date() } },
      ],
    },
  });
  return !!entitlement;
}

```

10.5 Access Key Management

Access keys provide API-level authentication for programmatic access:

```

// lib/access-keys/manager.ts
export async function createAccessKey(
  userId: string,
  options: AccessKeyOptions
): Promise<{ key: string; id: string }> {
  const rawKey = generateSecureKey(); // crypto.randomBytes(32)
  const hashedKey = await hashKey(rawKey);

  const record = await db.accessKey.create({
    data: {
      userId,
      hashedKey,
      name: options.name,
      scopes: options.scopes,
      expiresAt: options.expiresAt,
    },
  });

  // Return raw key ONCE – it is never stored or retrievable
  return { key: `aol_${rawKey}`, id: record.id };
}

```

⚠ WARNING Access keys are shown exactly once at creation time. They are stored as Argon2 hashes. If a user loses their key, it must be revoked and a new one issued. There is no recovery mechanism by design.

10.6 Inner Circle Membership

The Inner Circle is the premium membership tier with exclusive access:

```
interface InnerCircleMembership {
  userId: string;
  status: 'ACTIVE' | 'PAUSED' | 'CANCELLED' | 'LAPSED';
  tier: 'FOUNDING' | 'STANDARD';
  stripeSubscriptionId: string;
  startedAt: string;
  currentPeriodEnd: string;
  benefits: InnerCircleBenefit[];
}

enum InnerCircleBenefit {
  UNLIMITED_DIAGNOSTICS = 'UNLIMITED_DIAGNOSTICS',
  PRIORITY_SUPPORT = 'PRIORITY_SUPPORT',
  EARLY_ACCESS = 'EARLY_ACCESS',
  EXCLUSIVE_CONTENT = 'EXCLUSIVE_CONTENT',
  STRATEGY_SESSIONS = 'STRATEGY_SESSIONS',
  ENTERPRISE_FEATURES = 'ENTERPRISE_FEATURES',
}
```

PART IV — PRESENTATION LAYER

Chapter 11: Styling Architecture

11.1 Tailwind Configuration

The platform uses Tailwind CSS with a deeply customised institutional theme:

```
// tailwind.config.ts (key excerpts)
module.exports = {
  darkMode: 'class',
  content: [
    './app/**/*.ts,tsx,mdx',
    './components/**/*.ts,tsx',
    './lib/**/*.ts,tsx',
    './content/**/*.mdx',
  ],
  theme: {
    extend: {
      colors: {
        aol: {
          bg: 'var(--aol-bg)',
          ink: 'var(--aol-ink)',
          gold: 'var(--aol-gold)',
          muted: 'var(--aol-muted)',
        },
      },
    },
  },
}
```

```

    border: 'var(--aol-border)',
    surface: 'var(--aol-surface)',
    accent: 'var(--aol-accent)',
  },
},
fontFamily: {
  sans: ['Inter', 'system-ui', 'sans-serif'],
  serif: ['Cormorant Garamond', 'Georgia', 'serif'],
  mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
},
spacing: {
  'institutional': '1.618rem', // Golden ratio
},
letterSpacing: {
  'institutional': '0.05em',
  'institutional-wide': '0.1em',
  'institutional-tight': '0.02em',
},
},
},
};

```

11.2 Dark Mode (Class Strategy)

Dark mode is implemented via the `class` strategy — the `dark` class on `<html>` :

```

// lib/theme/provider.tsx
export function ThemeProvider({ children }: { children: React.ReactNode }) {
  return (
    <NextThemesProvider
      attribute="class"
      defaultTheme="system"
      enableSystem
      disableTransitionOnChange
    >
      {children}
    </NextThemesProvider>
  );
}

```

All colour values reference CSS variables, which flip between light and dark:

```

:root {
  --aol-bg: #fafaf8;
  --aol-ink: #1a1a1a;
  --aol-gold: #b8860b;
  --aol-muted: #6b7280;
  --aol-border: #e5e5e0;
  --aol-surface: #ffffff;
  --aol-accent: #1e3a5f;
}

```

```

}

.dark {
  --aol-bg: #0f0f0f;
  --aol-ink: #e8e8e8;
  --aol-gold: #d4a017;
  --aol-muted: #9ca3af;
  --aol-border: #2a2a2a;
  --aol-surface: #1a1a1a;
  --aol-accent: #4a90d9;
}

```

—— **KEY PRINCIPLE** *Never use raw colour values in components. Always reference the semantic CSS variables via Tailwind classes (`bg-aol-bg` , `text-aol-ink` , `border-aol-border`). This ensures dark mode works automatically.*

11.3 Component Variants (CVA Pattern)

Class Variance Authority (CVA) manages component variant styles:

```

// components/ui/button.tsx
import { cva, type VariantProps } from 'class-variance-authority';

const buttonVariants = cva(
  // Base styles
  'inline-flex items-center justify-center rounded-md font-medium transition-colors ' +
  'focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-aol-gold ' +
  'disabled:pointer-events-none disabled:opacity-50',
  {
    variants: {
      variant: {
        default: 'bg-aol-ink text-aol-bg hover:bg-aol-ink/90',
        destructive: 'bg-red-600 text-white hover:bg-red-700',
        outline: 'border border-aol-border bg-transparent hover:bg-aol-surface',
        ghost: 'hover:bg-aol-surface hover:text-aol-ink',
        link: 'text-aol-gold underline-offset-4 hover:underline',
        institutional: 'bg-aol-gold text-white font-serif tracking-institutional',
      },
      size: {
        sm: 'h-8 px-3 text-xs',
        default: 'h-10 px-4 py-2 text-sm',
        lg: 'h-12 px-6 text-base',
        icon: 'h-10 w-10',
      },
    },
    defaultVariants: {
      variant: 'default',
      size: 'default',
    },
  },
);

```



```

export interface ButtonProps
  extends React.ButtonHTMLAttributes<HTMLButtonElement>,
    VariantProps<typeof buttonVariants> {}

export const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
  ({ className, variant, size, ...props }, ref) => (
    <button
      className={cn(buttonVariants({ variant, size, className })))}
      ref={ref}
      {...props}
    />
  )
);

```

11.4 Radix UI Integration

Components are composed from Radix primitives → styled wrappers:

```

// Pattern: Radix Primitive → Styled Wrapper → Feature Component
// Example: Dialog

// 1. Import Radix primitive
import * as DialogPrimitive from '@radix-ui/react-dialog';

// 2. Create styled wrapper
const DialogOverlay = React.forwardRef<...>(({ className, ...props }, ref) => (
  <DialogPrimitive.Overlay
    ref={ref}
    className={cn(
      'fixed inset-0 z-50 bg-black/50 backdrop-blur-sm',
      'data-[state=open]:animate-in data-[state=closed]:animate-out',
      className
    )}
    {...props}
  />
));

// 3. Export composed component
export const Dialog = DialogPrimitive.Root;
export const DialogTrigger = DialogPrimitive.Trigger;
export const DialogContent = React.forwardRef<...>(({ children, ... }, ref) => (
  <DialogPortal>
    <DialogOverlay />
    <DialogPrimitive.Content ref={ref} className={cn(...)}>
      {children}
    </DialogPrimitive.Content>
  </DialogPortal>
));

```

11.5 Typography System

Three font families serve distinct purposes:

Font	Family	Usage
Inter	font-sans	Body text, UI elements, data tables
Cormorant Garamond	font-serif	Headings, pull quotes, institutional voice
JetBrains Mono	font-mono	Code, data values, technical references

Letter-spacing tokens:

Token	Value	Usage
tracking-institutional	0.05em	Standard institutional headings
tracking-institutional-wide	0.1em	All-caps labels, navigation
tracking-institutional-tight	0.02em	Large display headings

11.6 CSS Variables Reference

```
/* Core palette */
--aol-bg           /* Page background */
--aol-ink          /* Primary text */
--aol-gold         /* Accent / brand gold */
--aol-muted       /* Secondary text */
--aol-border       /* Borders and dividers */
--aol-surface      /* Card / elevated surface */
--aol-accent       /* Secondary accent (blue family) */

/* Extended palette */
--aol-success      /* Green – positive states */
--aol-warning      /* Amber – caution states */
--aol-error        /* Red – error / destructive */
--aol-info         /* Blue – informational */

/* Shadows */
--aol-shadow-sm    /* Subtle elevation */
--aol-shadow-md    /* Card elevation */
--aol-shadow-lg    /* Modal / dropdown elevation */
```

11.7 Responsive Patterns

Breakpoints follow Tailwind defaults with institutional additions:

```
screens: {
  'sm': '640px',
  'md': '768px',
```

```

    'lg': '1024px',
    'xl': '1280px',
    '2xl': '1536px',
    'institutional': '1440px', // Design system target width
  }

```

Standard responsive patterns:

```

{/* Stack on mobile, grid on desktop */}
<div className="flex flex-col gap-4 md:grid md:grid-cols-2 lg:grid-cols-3">

{/* Hide on mobile, show from md */}
<nav className="hidden md:flex">

{/* Full width mobile, contained desktop */}
<section className="px-4 md:px-8 lg:max-w-institutional lg:mx-auto">

```

11.8 Content Path Scanning

Tailwind scans the following paths for class usage:

```

content: [
  './app/**/*.{ts,tsx,mdx}',
  './components/**/*.{ts,tsx}',
  './lib/**/*.{ts,tsx}',
  './content/**/*.mdx',
]

```

⚠ WARNING If you add Tailwind classes in a new directory (e.g., a scripts folder that generates HTML), you **MUST** add it to the content array. Classes not in scanned paths will be purged from production builds.

Chapter 12: Component Architecture

12.1 Component Organization

```

components/
├─ ui/           ← Base primitives (Button, Input, Dialog, etc.)
├─ layout/       ← Shell, Header, Footer, Sidebar, Navigation
├─ features/     ← Domain-specific (DiagnosticCard, CampaignList)
├─ forms/        ← Form compositions (ContactForm, AssessmentForm)
├─ charts/       ← Data visualization components
├─ pdf/          ← PDF-specific React components
└─ shared/       ← Cross-cutting (ErrorBoundary, Loading, Empty states)

```

12.2 Composition Pattern

Radix Primitive → Styled Wrapper (ui/) → Feature Component (features/)

Example flow:

1. `@radix-ui/react-select` — raw accessible primitive
2. `components/ui/select.tsx` — styled with Tailwind + CVA variants
3. `components/features/domain-selector.tsx` — wired to domain data + handlers

——— **KEY PRINCIPLE** The `ui/` directory contains *ONLY* presentational components with no business logic. Feature components compose UI primitives and connect them to domain state. This separation must never be violated.

12.3 Design Tokens Flow

```
CSS Variables (globals.css)
→ Tailwind Config (theme.extend.colors)
→ CVA Variants (component-level)
→ Component Props (variant="institutional")
→ Rendered HTML (className="bg-aol-gold ...")
```

12.4 Animation (Framer Motion)

Standard animation patterns:

```
// Fade in on mount
<motion.div
  initial={{ opacity: 0, y: 8 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.3, ease: 'easeOut' }}
>

// Staggered list
<motion.ul>
  {items.map((item, i) => (
    <motion.li
      key={item.id}
      initial={{ opacity: 0, x: -12 }}
      animate={{ opacity: 1, x: 0 }}
      transition={{ delay: i * 0.05 }}
    />
  ))}
</motion.ul>

// Exit animation
<AnimatePresence mode="wait">
  {isVisible && (
    <motion.div
      exit={{ opacity: 0, scale: 0.95 }}
      transition={{ duration: 0.2 }}
    />
  )}
</AnimatePresence>
```

12.5 Icons (Lucide React)

```
import { ChevronRight, AlertTriangle, CheckCircle } from 'lucide-react';

// Standard sizing
<ChevronRight className="h-4 w-4" />           // Inline / small
<AlertTriangle className="h-5 w-5" />          // Default
<CheckCircle className="h-6 w-6" />           // Prominent
```

⚠ WARNING Do not use multiple icon libraries. Lucide React is the sole icon source. If a required icon does not exist in Lucide, create a custom SVG component in `components/icons/`.

12.6 Forms (React Hook Form + Zod)

```
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

const schema = z.object({
  email: z.string().email('Invalid email address'),
  message: z.string().min(10, 'At least 10 characters'),
});

type FormValues = z.infer<typeof schema>;

export function ContactForm() {
  const form = useForm<FormValues>({
    resolver: zodResolver(schema),
    defaultValues: { email: '', message: '' },
  });

  const onSubmit = async (data: FormValues) => {
    const res = await fetch('/api/contact', {
      method: 'POST',
      body: JSON.stringify(data),
    });
    // handle response
  };

  return (
    <Form {...form}>
      <form onSubmit={form.handleSubmit(onSubmit)}>
        <FormField name="email" control={form.control} render={...} />
        <FormField name="message" control={form.control} render={...} />
        <Button type="submit">Send</Button>
      </form>
    </Form>
  );
}
```

12.7 Tables (@tanstack/react-table)

```
import {
  useReactTable,
  getCoreRowModel,
  getSortedRowModel,
  getPaginationRowModel,
  flexRender,
} from '@tanstack/react-table';

const table = useReactTable({
  data,
  columns,
  getCoreRowModel: getCoreRowModel(),
  getSortedRowModel: getSortedRowModel(),
  getPaginationRowModel: getPaginationRowModel(),
  state: { sorting, pagination },
  onSortingChange: setSorting,
  onPaginationChange: setPagination,
});
```

12.8 Charts (Recharts)

```
import { ResponsiveContainer, BarChart, Bar, XAxis, YAxis, Tooltip } from 'recharts';

<ResponsiveContainer width="100%" height={300}>
  <BarChart data={domainScores}>
    <XAxis dataKey="domain" />
    <YAxis domain={[0, 100]} />
    <Tooltip />
    <Bar dataKey="score" fill="var(--aol-gold)" radius={[4, 4, 0, 0]} />
  </BarChart>
</ResponsiveContainer>
```

12.9 Toasts (Sonner)

```
import { toast } from 'sonner';

// Standard patterns
toast.success('Report generated successfully');
toast.error('Failed to save changes');
toast.loading('Generating PDF...');

// With action
toast('Session expired', {
  action: {
    label: 'Sign in',
```

```
    onClick: () => router.push('/auth/signin'),
  },
});
```

12.10 Command Palette (cmdk)

```
import { Command } from 'cmdk';

<Command>
  <Command.Input placeholder="Search commands..." />
  <Command.List>
    <Command.Group heading="Navigation">
      <Command.Item onSelect={() => router.push('/dashboard')}>
        Dashboard
      </Command.Item>
      <Command.Item onSelect={() => router.push('/diagnostics')}>
        Diagnostics
      </Command.Item>
    </Command.Group>
    <Command.Separator />
    <Command.Group heading="Actions">
      <Command.Item onSelect={createSession}>
        New Diagnostic Session
      </Command.Item>
    </Command.Group>
  </Command.List>
</Command>
```

Chapter 13: State Management

13.1 Zustand as Primary Store

```
// lib/stores/diagnostic-store.ts
import { create } from 'zustand';
import { devtools, persist, subscribeWithSelector } from 'zustand/middleware';

interface DiagnosticStore {
  // State
  sessions: DiagnosticSession[];
  activeSessionId: string | null;
  isLoading: boolean;

  // Actions
  setSessions: (sessions: DiagnosticSession[]) => void;
  setActiveSession: (id: string | null) => void;
  addResponse: (sessionId: string, response: Response) => void;
  reset: () => void;
}
```

```
export const useDiagnosticStore = create<DiagnosticStore>()(
  devtools(
    persist(
      subscribeWithSelector((set, get) => ({
        sessions: [],
        activeSessionId: null,
        isLoading: false,

        setSessions: (sessions) => set({ sessions }),
        setActiveSession: (id) => set({ activeSessionId: id }),
        addResponse: (sessionId, response) => set((state) => ({
          sessions: state.sessions.map((s) =>
            s.id === sessionId
              ? { ...s, responses: [...s.responses, response] }
              : s
          ),
        })),
        reset: () => set({ sessions: [], activeSessionId: null }),
      })),
      { name: 'diagnostic-store' }
    ),
    { name: 'DiagnosticStore' }
  )
);
```

Middleware stack (order matters):

1. `devtools` — outer (Redux DevTools integration)
2. `persist` — middle (localStorage/sessionStorage persistence)
3. `subscribeWithSelector` — inner (granular subscriptions)

13.2 When to Use Redux

Redux is reserved for complex multi-step workflows where Zustand's simplicity becomes a liability:

Use Case	Tool
Simple feature state (toggles, lists, UI)	Zustand
Server cache + mutations	React Query
Multi-step wizard with undo/redo	Redux
Complex inter-dependent state transitions	Redux
Shared state across unrelated features	Zustand
Theme / providers	Context API

—— **KEY PRINCIPLE** Default to Zustand. Only reach for Redux when you need middleware chains, action replay, or time-travel debugging for complex workflows. If you find yourself adding Redux for a simple feature, you are over-engineering.

13.3 React Query for Server State

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

// Query
export function useDiagnosticSession(id: string) {
  return useQuery({
    queryKey: ['diagnostic-session', id],
    queryFn: () => fetchSession(id),
    staleTime: 30_000, // 30 seconds
    retry: 2,
  });
}

// Mutation with optimistic update
export function useSubmitResponse() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: submitResponse,
    onMutate: async (newResponse) => {
      await queryClient.cancelQueries({ queryKey: ['diagnostic-session',
newResponse.sessionId] });
      const previous = queryClient.getQueryData(['diagnostic-session',
newResponse.sessionId]);

      queryClient.setQueryData(
        ['diagnostic-session', newResponse.sessionId],
        (old: any) => ({ ...old, responses: [...old.responses, newResponse] })
      );

      return { previous };
    },
    onError: (_err, newResponse, context) => {
      queryClient.setQueryData(
        ['diagnostic-session', newResponse.sessionId],
        context?.previous
      );
    },
    onSettled: (_data, _err, newResponse) => {
      queryClient.invalidateQueries({ queryKey: ['diagnostic-session',
newResponse.sessionId] });
    },
  });
}
```

13.4 Context API (Theme, Providers)

Context is used exclusively for:

- Theme (light/dark)
- Auth session (via NextAuth `SessionProvider`)
- Feature flags
- Locale / i18n

```
// Providers composition (app/providers.tsx)
export function Providers({ children }: { children: React.ReactNode }) {
  return (
    <SessionProvider>
      <ThemeProvider>
        <QueryClientProvider client={queryClient}>
          <TooltipProvider>
            {children}
          </TooltipProvider>
        </QueryClientProvider>
      </ThemeProvider>
    </SessionProvider>
  );
}
```

13.5 Server Components and Data Flow

Next.js Server Components eliminate the need for client state in many cases:

```
// app/diagnostics/[id]/page.tsx (Server Component – no "use client")
export default async function DiagnosticPage({ params }: { params: { id: string } }) {
  const session = await getAuthSession();
  if (!session) redirect('/auth/signin');

  const diagnostic = await db.diagnosticSession.findUnique({
    where: { id: params.id, userId: session.user.id },
    include: { responses: true, reports: true },
  });

  if (!diagnostic) notFound();

  // No client state needed – data flows directly from DB to render
  return <DiagnosticDetail session={diagnostic} />;
}
```

—— **KEY PRINCIPLE** If data can be fetched at the server component level and does not need real-time updates or user interaction state, do NOT introduce client-side state management. Server Components are the default. Client components are the exception.

PART V — API & INTEGRATION LAYER

Chapter 14: API Architecture

14.1 Route Organization by Domain

```
app/api/  
├─ auth/           ← NextAuth routes + custom auth endpoints  
├─ alignment/      ← Campaign, organisation, enterprise assessment  
├─ audit/          ← Audit log queries  
├─ decision/       ← Strategy room decisions  
├─ diagnostics/    ← Sessions, responses, reports, artifacts  
├─ vault/          ← Secure document storage  
├─ admin/          ← Admin-only operations  
├─ cron/           ← Scheduled job triggers  
├─ analytics/      ← Event tracking, dashboards  
├─ search/         ← Algolia query proxy  
├─ checkout/       ← Stripe checkout + webhooks  
├─ contact/        ← Public contact form  
├─ pdf/            ← PDF generation triggers  
└─ v2/             ← Versioned API (breaking changes)
```

14.2 Standard Handler Pattern

Every API route follows this structure:

```
// app/api/[domain]/[resource]/route.ts  
import { NextRequest } from 'next/server';  
import { z } from 'zod';  
import { getAuthSession } from '@lib/auth';  
import { rateLimit } from '@lib/rate-limit';  
import { json, unauthorized, forbidden, badRequest } from '@lib/api/responses';  
  
const requestSchema = z.object({  
  title: z.string().min(1).max(200),  
  description: z.string().optional(),  
});  
  
export async function POST(req: NextRequest) {  
  // 1. Rate limiting  
  const rateLimitResult = await rateLimit(req);  
  if (!rateLimitResult.ok) {  
    return json({ ok: false, error: 'Rate limit exceeded', code: 429 });  
  }  
  
  // 2. Authentication  
  const session = await getAuthSession();  
  if (!session) return unauthorized();  
  
  // 3. Authorization (role/permission checks)  
  if (!hasPermission(session.user, 'create:campaign')) {  
    return forbidden();  
  }  
}
```

```

}

// 4. Input validation
const body = await req.json();
const parsed = requestSchema.safeParse(body);
if (!parsed.success) {
  return badRequest(parsed.error.flatten().fieldErrors);
}

// 5. Business logic
try {
  const result = await createCampaign(session.user.id, parsed.data);
  return json({ ok: true, data: result, code: 201 });
} catch (error) {
  return json({ ok: false, error: 'Failed to create campaign', code: 500 });
}
}

```

14.3 Error Handling

Standard response format — every response matches this shape:

```

interface ApiResponse<T = unknown> {
  ok: boolean;
  data?: T;
  error?: string;
  code: number;
}

// Helper functions (lib/api/responses.ts)
export function json<T>(response: ApiResponse<T>): NextResponse {
  return NextResponse.json(response, { status: response.code });
}

export function unauthorized() {
  return json({ ok: false, error: 'Authentication required', code: 401 });
}

export function forbidden() {
  return json({ ok: false, error: 'Insufficient permissions', code: 403 });
}

export function badRequest(errors: Record<string, string[]>) {
  return json({ ok: false, error: 'Validation failed', data: errors, code: 422 });
}

export function notFound(resource?: string) {
  return json({ ok: false, error: `${resource ?? 'Resource'} not found`, code: 404 });
}

```

⚠ WARNING Never return raw error messages from caught exceptions to the client. Internal errors must be logged server-side and a generic message returned. Leaking stack traces or database errors is a security vulnerability.

14.4 Rate Limiting Implementation

```
// lib/rate-limit.ts
import { Ratelimit } from '@upstash/ratelimit';
import { Redis } from '@upstash/redis';

// Primary: Upstash Redis (production)
const upstashLimiter = process.env.UPSTASH_REDIS_REST_URL
  ? new Ratelimit({
      redis: Redis.fromEnv(),
      limiter: Ratelimit.slidingWindow(100, '15 m'),
      analytics: true,
    })
  : null;

// Fallback: In-memory (development / when Redis unavailable)
const memoryStore = new Map<string, { count: number; resetAt: number }>();

export async function rateLimit(req: NextRequest): Promise<{ ok: boolean; remaining: number }> {
  const identifier = getIdentifier(req); // IP or user ID

  if (upstashLimiter) {
    const result = await upstashLimiter.limit(identifier);
    return { ok: result.success, remaining: result.remaining };
  }

  // In-memory fallback
  const now = Date.now();
  const window = 15 * 60 * 1000; // 15 minutes
  const entry = memoryStore.get(identifier);

  if (!entry || now > entry.resetAt) {
    memoryStore.set(identifier, { count: 1, resetAt: now + window });
    return { ok: true, remaining: 99 };
  }

  entry.count++;
  if (entry.count > 100) {
    return { ok: false, remaining: 0 };
  }

  return { ok: true, remaining: 100 - entry.count };
}
```

Rate limit: **100 requests per 15 minutes** per identifier (IP or authenticated user ID).

14.5 CORS Configuration

```
// middleware.ts (or lib/api/cors.ts)
const ALLOWED_ORIGINS = [
  'https://abrahamoflondon.com',
  'https://www.abrahamoflondon.com',
  'https://app.abrahamoflondon.com',
  process.env.NODE_ENV === 'development' && 'http://localhost:3000',
].filter(Boolean) as string[];

export function corsHeaders(origin: string | null): HeadersInit {
  const isAllowed = origin && ALLOWED_ORIGINS.includes(origin);
  return {
    'Access-Control-Allow-Origin': isAllowed ? origin : '',
    'Access-Control-Allow-Methods': 'GET, POST, PUT, PATCH, DELETE, OPTIONS',
    'Access-Control-Allow-Headers': 'Content-Type, Authorization, X-CSRF-Token',
    'Access-Control-Max-Age': '86400',
  };
}
```

—— **KEY PRINCIPLE** CORS is a whitelist — only explicitly approved origins may call API routes. The wildcard (`*`) is never used in production. Development origins are stripped at build time via environment checks.

14.6 Authentication in API Routes

```
// lib/auth/session.ts
import { getServerSession } from 'next-auth';
import { authOptions } from '@app/api/auth/[...nextauth]/options';

export async function getAuthSession() {
  return getServerSession(authOptions);
}

// Usage in route handlers
export async function GET(req: NextRequest) {
  const session = await getAuthSession();
  if (!session?.user?.id) return unauthorized();

  // session.user.id is now guaranteed to be a string
  const data = await fetchUserData(session.user.id);
  return json({ ok: true, data, code: 200 });
}
```

14.7 API Versioning

Breaking changes are served under `/api/v2/` :

```
app/api/v2/
├─ diagnostics/    ← New response shapes, removed deprecated fields
```

```
└─ alignment/      ← Restructured campaign model
```

Versioning rules:

- Additive changes (new fields, new endpoints) → no version bump
- Removing fields, changing types, restructuring → new version
- Old versions are maintained for 6 months after deprecation notice

Chapter 15: External Integrations

15.1 Resend (Email)

```
// lib/email/client.ts
import { Resend } from 'resend';

export const resend = new Resend(process.env.RESEND_API_KEY);

// Email types
enum EmailType {
  CONTACT = 'CONTACT',
  INNER_CIRCLE = 'INNER_CIRCLE',
  INVITE = 'INVITE',
  ENTERPRISE = 'ENTERPRISE',
  SYSTEM = 'SYSTEM',
  TRANSACTIONAL = 'TRANSACTIONAL',
}

// Template system
// lib/email/templates/
// └─ contact.tsx      ← React Email template
// └─ inner-circle.tsx
// └─ invite.tsx
// └─ enterprise.tsx
// └─ system.tsx
// └─ transactional.tsx

export async function sendEmail(options: SendEmailOptions): Promise<void> {
  const template = getTemplate(options.type);
  const html = render(template(options.data));

  await resend.emails.send({
    from: options.from ?? 'Abraham of London <hello@abrahamoflondon.com>',
    to: options.to,
    subject: options.subject,
    html,
    tags: [{ name: 'type', value: options.type }],
  });
}
```

15.2 Stripe (Payments)

Key integration points:

Component	Purpose
lib/stripe/client.ts	Stripe SDK instance
lib/stripe/webhooks.ts	Event handlers by type
lib/stripe/customers.ts	Customer create/retrieve
lib/stripe/subscriptions.ts	Subscription lifecycle
app/api/checkout/webhook/route.ts	Webhook receiver

Webhook events handled:

```
const HANDLED_EVENTS = [  
  'checkout.session.completed',  
  'customer.subscription.created',  
  'customer.subscription.updated',  
  'customer.subscription.deleted',  
  'invoice.payment_succeeded',  
  'invoice.payment_failed',  
] as const;
```

15.3 Algolia (Search)

```
// lib/search/algolia.ts  
import algoliasearch from 'algoliasearch';  
  
const client = algoliasearch(  
  process.env.ALGOLIA_APP_ID!,  
  process.env.ALGOLIA_ADMIN_KEY!  
);  
  
// Indexing strategy – separate indices by content type  
export const indices = {  
  content: client.initIndex('content'),  
  lexicon: client.initIndex('lexicon'),  
  diagnostics: client.initIndex('diagnostics'),  
};  
  
// Query pattern (via API proxy to protect admin key)  
export async function search(query: string, options?: SearchOptions) {  
  const searchClient = algoliasearch(  
    process.env.ALGOLIA_APP_ID!,  
    process.env.ALGOLIA_SEARCH_KEY! // Search-only key for client  
  );  
  return searchClient.initIndex(options?.index ?? 'content').search(query, {
```



```

    hitsPerPage: options?.limit ?? 20,
    filters: options?.filters,
  });
}

```

15.4 Anthropic / OpenAI (AI)

```

// lib/ai/client.ts
import Anthropic from '@anthropic-ai/sdk';

const anthropic = new Anthropic({
  apiKey: process.env.ANTHROPIC_API_KEY!,
});

// Standard prompt pattern
export async function generateNarrative(
  context: NarrativeContext
): Promise<string> {
  const message = await anthropic.messages.create({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 2048,
    system: SYSTEM_PROMPT,
    messages: [
      {
        role: 'user',
        content: buildNarrativePrompt(context),
      },
    ],
  });

  return extractTextContent(message);
}

```

⚠ WARNING AI-generated content must always be clearly marked as such in user-facing surfaces. Never present AI output as if it were written by a human practitioner. The narrative engine produces drafts — final reports require human review before delivery.

15.5 Neon (Database)

```

// lib/db/client.ts
import { PrismaClient } from '@prisma/client';
import { neonConfig } from '@neondatabase/serverless';

// Serverless adapter for edge/serverless environments
neonConfig.fetchConnectionCache = true;

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const db = globalForPrisma.prisma ?? new PrismaClient({

```

```

datasources: {
  db: { url: process.env.DATABASE_URL },
},
log: process.env.NODE_ENV === 'development' ? ['query', 'warn', 'error'] : ['error'],
});

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = db;

```

Connection pooling is managed by Neon's serverless driver — no PgBouncer required.

15.6 Upstash (Redis / Rate Limiting)

```

// lib/redis/client.ts
import { Redis } from '@upstash/redis';

export const redis = Redis.fromEnv();
// Expects UPSTASH_REDIS_REST_URL and UPSTASH_REDIS_REST_TOKEN

// Usage patterns
await redis.set(`session:${id}`, data, { ex: 3600 });
await redis.get(`session:${id}`);
await redis.del(`session:${id}`);
await redis.incr(`counter:${key}`);

```

15.7 Vercel Analytics

```

// Event tracking
import { track } from '@vercel/analytics';

// Client-side
track('diagnostic_started', { type: 'individual' });
track('campaign_created', { participants: 5 });
track('pdf_downloaded', { format: 'diagnostic_report' });

// Server-side (via API)
import { Analytics } from '@vercel/analytics/server';
Analytics.track('payment_completed', { amount: 9900, currency: 'GBP' });

```

Chapter 16: Serverless Functions (Netlify)

16.1 Available Functions

Function	Trigger	Purpose
contact	HTTP	Process contact form submissions
download	HTTP	Secure file download with signed URLs

subscribe	HTTP	Newsletter / mailing list subscription
health-check	HTTP/Cron	Platform health verification
cleanup	Cron	Expired session and orphan cleanup
test-email	HTTP	Email delivery verification (staging only)
ping	HTTP	Simple availability check

16.2 Function Architecture

```

netlify/functions/
├─ contact.ts
├─ download.ts
├─ subscribe.ts
├─ health-check.ts
├─ cleanup.ts
├─ test-email.ts
├─ ping.ts
└─ _shared/
    ├─ _utils.ts      ← Common utilities (validation, response builders)
    └─ _email.ts      ← Email sending (Resend integration)

```

Shared modules (prefixed with `_`) are not deployed as individual functions — they are imported by function handlers:

```

// netlify/functions/_shared/_utils.ts
export function createResponse(statusCode: number, body: unknown) {
  return {
    statusCode,
    headers: {
      'Content-Type': 'application/json',
      'Access-Control-Allow-Origin': process.env.ALLOWED_ORIGIN ?? '*',
    },
    body: JSON.stringify(body),
  };
}

export function validateRequest(event: HandlerEvent, schema: z.ZodSchema) {
  const body = JSON.parse(event.body ?? '{}');
  return schema.safeParse(body);
}

```

16.3 Scheduled Functions (Cron Patterns)

```

# netlify.toml

[functions."health-check"]
schedule = "*/5 * * * *" # Every 5 minutes

```

```
[functions."cleanup"]
schedule = "0 3 * * *"      # Daily at 03:00 UTC
```

Scheduled function signature:

```
// netlify/functions/cleanup.ts
import { schedule } from '@netlify/functions';

export const handler = schedule('0 3 * * *', async (event) => {
  const expiredSessions = await cleanExpiredSessions();
  const orphanedArtifacts = await cleanOrphanedArtifacts();

  console.log(`Cleanup: ${expiredSessions} sessions, ${orphanedArtifacts} artifacts`);

  return {
    statusCode: 200,
  };
});
```

16.4 Configuration

```
# netlify.toml

[build]
  command = "npm run build"
  publish = ".next"

[functions]
  directory = "netlify/functions"
  node_bundler = "esbuild"
  external_node_modules = ["@prisma/client", "prisma"]

[functions."**"]
  timeout = 30
```

16.5 Bundle Optimization

Prisma is externalised to avoid bundling the query engine into each function:

```
external_node_modules = ["@prisma/client", "prisma"]
```

This ensures:

- Function bundles remain small (< 50MB limit)
- Prisma's native query engine is loaded at runtime
- Cold start times are minimised

—— **KEY PRINCIPLE** *Netlify functions have a 50MB bundle limit and 10-second default timeout (30s configured). Keep functions lean — heavy computation belongs in background functions or API routes on the primary platform.*

16.6 Environment Variable Pass-Through

All environment variables set in the Netlify dashboard are available to functions. Key variables required:

DATABASE_URL	– Neon connection string
RESEND_API_KEY	– Email sending
STRIPE_SECRET_KEY	– Payment verification
STRIPE_WEBHOOK_SECRET	– Webhook signature validation
UPSTASH_REDIS_REST_URL	– Rate limiting
UPSTASH_REDIS_REST_TOKEN	– Rate limiting auth
ALGOLIA_APP_ID	– Search
ALGOLIA_ADMIN_KEY	– Indexing
ALLOWED_ORIGIN	– CORS

PART VI — PDF PIPELINE

Chapter 17: PDF Architecture

17.1 Five Generation Paths

The platform maintains five PDF generation paths, each optimised for different use cases:

Path	Technology	Use Case	Pros	Cons
React PDF	@react-pdf/renderer	Diagnostic reports, data-heavy documents	Full React component model, dynamic layouts	No CSS support, limited styling
jsPDF	jspdf + jspdf-autotable	Simple certificates, receipts	Fast, no server deps, client-side capable	Limited layout control
Puppeteer	puppeteer	Complex visual layouts, branded content	Full CSS/HTML support	Heavy, requires Chrome, slow
LibreOffice	libreoffice --headless	DOCX → PDF conversion	Handles complex Word documents	Requires system binary, slow
md-to-pdf	md-to-pdf	Markdown content pages, documentation	Simple input format, consistent output	Limited styling

17.2 When to Use Which Path

Decision tree:

Is the source Markdown?
→ Yes: md-to-pdf

→ No: Continue

Is the source a .docx file?

→ Yes: LibreOffice

→ No: Continue

Does it need complex CSS layouts (grids, gradients, precise positioning)?

→ Yes: Puppeteer

→ No: Continue

Is it data-driven with tables, charts, dynamic sections?

→ Yes: React PDF

→ No: Continue

Is it a simple single-page document (certificate, receipt)?

→ Yes: jsPDF

→ No: React PDF (default)

—— **KEY PRINCIPLE** *React PDF is the default. Only deviate when the content requirements demand a different renderer. Each additional path increases maintenance burden — justify it.*

17.3 Unified PDF Generator

```
// scripts/unified-pdf-generator.ts
// The master orchestrator for all PDF generation

interface GenerationRequest {
  source: string;           // Source file or data
  renderer: RendererType;   // REACT_PDF | JSPDF | PUPPETEER | LIBREOFFICE | MD_TO_PDF
  outputPath: string;       // Destination path
  options: RendererOptions;
  metadata: PDFMetadata;
}

export async function generatePDF(request: GenerationRequest): Promise<GenerationResult> {
  // 1. Validate request
  validateRequest(request);

  // 2. Select renderer
  const renderer = getRenderer(request.renderer);

  // 3. Pre-process source
  const processed = await preProcess(request.source, request.options);

  // 4. Generate
  const buffer = await renderer.generate(processed, request.options);

  // 5. Post-process (watermark, metadata, fingerprint)
  const final = await postProcess(buffer, request.metadata);

  // 6. Write output
```

```

    await writeOutput(final, request.outputPath);

    // 7. Register in manifest
    await updateManifest(request.outputPath, request.metadata);

    return {
      path: request.outputPath,
      size: final.byteLength,
      renderer: request.renderer,
      generatedAt: new Date().toISOString(),
    };
  }
}

```

17.4 Registry System

The PDF registry tracks all generated PDFs:

```

// scripts/build-pdf-registry-generated.ts → lib/pdf/pdf-registry.generated.ts

// Registry entry
interface PDFRegistryEntry {
  id: string;
  filename: string;
  path: string;
  renderer: RendererType;
  category: string; // diagnostic, alignment, certificate, content
  generatedAt: string;
  size: number;
  hash: string; // SHA-256 of content
  metadata: PDFMetadata;
  governance: GovernanceStatus;
}

// Generated registry file (auto-generated – do not edit manually)
// lib/pdf/pdf-registry.generated.ts
export const PDF_REGISTRY: Map<string, PDFRegistryEntry> = new Map([
  ['diagnostic-report-001', { ... }],
  ['alignment-summary-001', { ... }],
  // ... 198 entries
]);

```

⚠ WARNING The registry file `pdf-registry.generated.ts` is AUTO-GENERATED by `build-pdf-registry-generated.ts`. Never edit it manually. Run `pnpm pdf:registry` to regenerate after adding or modifying PDFs.

17.5 Manifest and Audit Files

```

public/pdf/
├─ manifest.json      ← Complete inventory of all PDFs
├─ audit-report.json  ← Last audit results

```

```
| governance-report.json ← Governance compliance status
| [generated PDFs]
```

Manifest structure:

```
{
  "version": "1.0.0",
  "generatedAt": "2026-04-27T00:00:00Z",
  "totalFiles": 198,
  "categories": {
    "diagnostic": 45,
    "alignment": 32,
    "certificate": 18,
    "content": 103
  },
  "entries": [...]
}
```

17.6 Governance Rules

All PDFs must comply with these governance rules:

Rule	Requirement	Validation
Naming	[category]-[descriptor]-[version].[ext]	Regex pattern match
Size	Maximum 10MB per file	File size check
Header	Must contain institutional header with logo	First-page analysis
Metadata	Title, author, creation date required	PDF metadata extraction
Fonts	Only approved font families (Inter, Cormorant Garamond, JetBrains Mono)	Font embedding check
Colour	Must use institutional palette	Colour sampling
Hash	SHA-256 recorded in registry	Integrity verification

```
// lib/pdf/governance.ts
export function validateGovernance(entry: PDFRegistryEntry): GovernanceResult {
  const violations: GovernanceViolation[] = [];

  if (!NAMING_PATTERN.test(entry.filename)) {
    violations.push({ rule: 'naming', message: 'Filename does not match pattern' });
  }

  if (entry.size > MAX_PDF_SIZE) {
    violations.push({ rule: 'size', message: `Exceeds ${MAX_PDF_SIZE / 1024 / 1024}MB limit`
  });
}
```



```
// ... additional checks

return {
  compliant: violations.length === 0,
  violations,
  checkedAt: new Date().toISOString(),
};
}
```

Chapter 18: PDF Operations

18.1 Commands Reference

Command	Script	Purpose
<code>pnpm pdf:generate</code>	<code>scripts/unified-pdf-generator.ts</code>	Generate PDFs from source files
<code>pnpm pdf:registry</code>	<code>scripts/build-pdf-registry-generated.ts</code>	Rebuild the PDF registry
<code>pnpm pdf:audit</code>	<code>scripts/pdf-audit.ts</code>	Run full audit (duplicates, links, governance)
<code>pnpm pdf:audit:duplicates</code>	<code>scripts/pdf-audit-duplicates.ts</code>	Find duplicate PDFs by hash
<code>pnpm pdf:audit:canonicals</code>	<code>scripts/pdf-audit-canonicals.ts</code>	Verify canonical references
<code>pnpm pdf:audit:links</code>	<code>scripts/pdf-audit-links.ts</code>	Check internal/external links
<code>pnpm pdf:audit:governance</code>	<code>scripts/pdf-audit-governance.ts</code>	Validate governance compliance
<code>pnpm pdf:repair</code>	<code>scripts/pdf-repair.ts</code>	Detect and fix corrupt/empty PDFs
<code>pnpm pdf:repair:detect</code>	<code>scripts/pdf-repair-detect.ts</code>	Identify corrupt files only
<code>pnpm pdf:repair:fix</code>	<code>scripts/pdf-repair-fix.ts</code>	Attempt automatic repair
<code>pnpm pdf:manifest</code>	<code>scripts/pdf-manifest.ts</code>	Regenerate manifest.json
<code>pnpm pdf:validate</code>	<code>scripts/pdf-validate.ts</code>	Validate specific PDF(s)
<code>pnpm pdf:clean</code>	<code>scripts/pdf-clean.ts</code>	Remove orphaned/invalid PDFs
<code>pnpm pdf:stats</code>	<code>scripts/pdf-stats.ts</code>	Print registry statistics

18.2 Generation Workflow

scan → generate → validate → register

1. **Scan** — Identify source files requiring PDF generation
2. **Generate** — Route through appropriate renderer
3. **Validate** — Check governance compliance
4. **Register** — Add to registry and manifest

```
# Full pipeline
pnpm pdf:generate --all
pnpm pdf:audit:governance
pnpm pdf:registry
pnpm pdf:manifest

# Single file
pnpm pdf:generate --source content/reports/diagnostic-001.mdx --renderer react-pdf
pnpm pdf:validate --file public/pdf/diagnostic-report-001.pdf
```

18.3 Audit Workflow

```
# Complete audit
pnpm pdf:audit

# Individual checks
pnpm pdf:audit:duplicates    # SHA-256 hash comparison
pnpm pdf:audit:canonicals    # Canonical URL verification
pnpm pdf:audit:links         # Dead link detection
pnpm pdf:audit:governance    # Rule compliance
```

Audit output writes to `public/pdf/audit-report.json` :

```
{
  "auditedAt": "2026-04-27T00:00:00Z",
  "totalFiles": 198,
  "passed": 192,
  "failed": 6,
  "issues": [
    {
      "file": "diagnostic-report-047.pdf",
      "rule": "size",
      "severity": "warning",
      "message": "File size 9.8MB approaching 10MB limit"
    }
  ]
}
```

18.4 Repair Workflow

```
# Detect issues
pnpm pdf:repair:detect
```

```
# Output:
# Found 3 corrupt files:
#   - certificate-legacy-002.pdf (0 bytes)
#   - alignment-summary-015.pdf (truncated, 2.1KB)
#   - content-framework-008.pdf (invalid header)

# Attempt automatic repair
pnpm pdf:repair:fix

# Repair strategies:
# 1. Empty files → regenerate from source
# 2. Truncated → regenerate from source
# 3. Invalid header → repair metadata
# 4. Unfixable → flag for manual intervention
```

18.5 Font Management

```
// lib/pdf/font-registry.ts
export const FONT_REGISTRY = {
  Inter: {
    family: 'Inter',
    weights: {
      regular: '/fonts/Inter-Regular.woff2',
      medium: '/fonts/Inter-Medium.woff2',
      semibold: '/fonts/Inter-SemiBold.woff2',
      bold: '/fonts/Inter-Bold.woff2',
    },
    fallback: 'Helvetica',
  },
  'Cormorant Garamond': {
    family: 'Cormorant Garamond',
    weights: {
      regular: '/fonts/CormorantGaramond-Regular.woff2',
      medium: '/fonts/CormorantGaramond-Medium.woff2',
      semibold: '/fonts/CormorantGaramond-SemiBold.woff2',
      bold: '/fonts/CormorantGaramond-Bold.woff2',
    },
    fallback: 'Times-Roman',
  },
  'JetBrains Mono': {
    family: 'JetBrains Mono',
    weights: {
      regular: '/fonts/JetBrainsMono-Regular.woff2',
      medium: '/fonts/JetBrainsMono-Medium.woff2',
      bold: '/fonts/JetBrainsMono-Bold.woff2',
    },
    fallback: 'Courier',
  },
} as const;
```

```
// Register fonts for React PDF
import { Font } from '@react-pdf/renderer';

export function registerPDFFonts(): void {
  Object.values(FONT_REGISTRY).forEach((font) => {
    Font.register({
      family: font.family,
      fonts: Object.entries(font.weights).map(([weight, src]) => ({
        src,
        fontWeight: weight,
      })),
    });
  });
}

```

18.6 Watermarking and Fingerprinting

```
// lib/pdf/watermark.ts
interface WatermarkOptions {
  text?: string; // Visible watermark text
  fingerprint: string; // Unique identifier embedded invisibly
  opacity?: number; // 0-1 for visible watermark
  position?: 'diagonal' | 'footer' | 'header';
}

export async function applyWatermark(
  pdfBuffer: Buffer,
  options: WatermarkOptions
): Promise<Buffer> {
  const pdfDoc = await PDFDocument.load(pdfBuffer);

  // Invisible fingerprint – embedded in document metadata
  pdfDoc.setCustomProperty('aol:fingerprint', options.fingerprint);
  pdfDoc.setCustomProperty('aol:generated', new Date().toISOString());

  // Visible watermark (if text provided)
  if (options.text) {
    const pages = pdfDoc.getPages();
    for (const page of pages) {
      page.drawText(options.text, {
        x: page.getWidth() / 2,
        y: page.getHeight() / 2,
        size: 48,
        opacity: options.opacity ?? 0.08,
        rotate: degrees(45),
        color: rgb(0.5, 0.5, 0.5),
      });
    }
  }
}

```

```
    return Buffer.from(await pdfDoc.save());
}
```

—— **KEY PRINCIPLE** Every PDF generated by the platform carries an invisible fingerprint. This enables tracing leaked documents back to the specific user and generation event. The fingerprint is a SHA-256 hash of `userId + generationTimestamp + documentId`.

18.7 Enterprise PDF Generation

Enterprise PDFs are generated via API routes and Netlify functions:

```
// app/api/pdf/enterprise/route.ts
export async function POST(req: NextRequest) {
  const session = await getAuthSession();
  if (!session) return unauthorized();

  const body = await req.json();
  const { campaignId, format } = enterprisePdfSchema.parse(body);

  // Verify entitlement
  if (!await hasEntitlement(session.user.id, 'TIER', 'ENTERPRISE')) {
    return forbidden();
  }

  // Generate asynchronously – return job ID
  const jobId = await queuePdfGeneration({
    type: 'enterprise_assessment',
    campaignId,
    format,
    userId: session.user.id,
    renderer: 'REACT_PDF',
    watermark: {
      fingerprint: generateFingerprint(session.user.id, campaignId),
    },
  });

  return json({ ok: true, data: { jobId }, code: 202 });
}
```

Netlify function for background generation:

```
// netlify/functions/generate-pdf.ts
import { Handler } from '@netlify/functions';

export const handler: Handler = async (event) => {
  const { jobId } = JSON.parse(event.body ?? '{}');
  const job = await getJob(jobId);

  const result = await generatePDF({
    source: job.source,
```

```
    renderer: job.renderer,
    outputPath: `/tmp/${job.id}.pdf`,
    options: job.options,
    metadata: job.metadata,
  });

  // Upload to S3
  const url = await uploadToStorage(result.path, `pdfs/${job.id}.pdf`);

  // Update job status
  await updateJob(jobId, { status: 'COMPLETE', url });

  return { statusCode: 200 };
};
```

———— **KEY PRINCIPLE** *This manual is the single source of truth for engineering decisions at Abraham of London. When in doubt, consult this document. When the document is silent, escalate to the Technical Director. When the document conflicts with intuition, the document wins until it is formally amended.*

End of Part B — Engineering Manual

ABRAHAM OF LONDON

ENGINEERING MANUAL — PART C

Quality, Operations, Security & Developer Operations

Document Control

Field	Detail
Classification	Internal — Engineering Staff Only
Version	1.0
Effective Date	April 2026
Review Cycle	Quarterly
Custodian	Engineering Lead / Office of the Founder
Next Review	July 2026
Depends On	Engineering Manual Parts A & B

PART VII — QUALITY & TESTING

Chapter 19: Testing Strategy

19.1 Framework Stack

Tool	Role	Environment
Vitest 4.1.2	Primary unit/integration runner	Node
Jest (jsdom)	Legacy fallback for DOM-dependent tests	jsdom
Playwright	End-to-end browser testing	Chromium/Firefox/WebKit

———— **KEY PRINCIPLE**

Tests exist to protect institutional logic — deal fusion, predictive intelligence, entitlement enforcement, and governance audit trails. If a function affects revenue or security, it must be tested. No exceptions.

19.2 Configuration — vitest.config.ts

```
import { defineConfig } from 'vitest/config';
import path from 'path';

export default defineConfig({
  test: {
    environment: 'node',
    globals: true,
    alias: {
      '@': path.resolve(__dirname, './'),
    },
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html'],
      include: ['lib/ai/**/*.ts'],
      exclude: [
        'node_modules/**',
        'lib/ai/__tests__/**',
      ],
      thresholds: {
        lines: 90,
        functions: 95,
        branches: 85,
        statements: 90,
      },
    },
    maxConcurrency: 5,
    testTimeout: 10000,
  },
});
```

19.3 Test Locations

Pattern	Purpose
lib/**/*.test.ts	Domain logic unit tests

tests/	Integration and E2E suites
tests/performance/	Performance benchmarks
tests/integration/	Cross-module integration
Co-located *.test.ts	Small utilities tested alongside source

19.4 Coverage Targets

Scope	Target	Rationale
lib/ai/	90% lines, 95% functions, 85% branches	Revenue-critical AI logic
Global	80% lines	Institutional baseline
lib/predictive/	85% lines	Decision intelligence accuracy
lib/decision/	85% lines	Governance enforcement

19.5 Test Categories

Unit Tests — Isolated function logic. No network, no database, no file system. Mock all external dependencies.

```
// lib/ai/__tests__/deal-fusion.test.ts
describe('DealFusion', () => {
  it('calculates alignment score correctly', () => {
    const result = calculateAlignment(mockInput);
    expect(result.score).toBeGreaterThan(0.7);
    expect(result.band).toBe('ALIGNED');
  });
});
```

Integration Tests — Multiple modules working together. Database may be involved (use test DB).

```
pnpm test:integration # vitest run tests/integration --runInBand
```

E2E Tests — Full browser automation via Playwright. Tests critical user flows: login, assessment submission, report generation.

Performance Tests — Benchmark critical paths. Run in isolation to avoid flaky timing.

```
pnpm test:performance # vitest run tests/performance --runInBand
```

19.6 What to Test

Category	Test?	Notes
Domain logic (lib/ai/, lib/predictive/, lib/decision/)	Always	Core revenue engine
API routes (app/api/)	Always	Input validation, auth, response shape
Utilities (lib/utils/, lib/helpers/)	Always	Shared code = shared risk

UI components	Playwright	Visual/interaction testing only
Database queries	Integration	Test with real DB, seed fixtures
Security functions	Always	Rate limiting, auth, encryption

19.7 What NOT to Test

⚠ PROHIBITION

Do not write tests for:

- **Generated code** — Contentlayer output, Prisma client, auto-generated types
- **Third-party wrappers** — Thin wrappers around `@anthropic-ai/sdk`, Stripe SDK, etc.
- **Contentlayer output** — Schema-validated at build time; runtime testing is redundant
- **Static configuration** — `next.config.mjs`, `tailwind.config.ts`, etc.

Testing generated code wastes CI time and creates false confidence.

19.8 Commands Reference

Command	Action
<code>pnpm test</code>	Run Vitest (all tests, watch mode)
<code>pnpm test:ui</code>	Launch Vitest UI in browser
<code>pnpm test:watch</code>	Watch specific test files
<code>pnpm test:coverage</code>	Run with V8 coverage
<code>pnpm test:coverage:html</code>	Coverage with HTML report
<code>pnpm test:all</code>	<code>test + check:all + pdf:audit</code>
<code>pnpm test:integration</code>	Integration suite (sequential)
<code>pnpm test:performance</code>	Performance benchmarks (sequential)
<code>pnpm test:deal-fusion</code>	Single domain test file
<code>pnpm test:predictive</code>	Predictive intelligence suite
<code>pnpm test:decision</code>	Decision engine suite
<code>pnpm benchmark:predictive</code>	TSX benchmark runner
<code>pnpm validate</code>	<code>typecheck + lint + test</code>

Chapter 20: Code Quality

20.1 ESLint Configuration (Flat Config — ESLint 10)

The project uses `eslint.config.mjs` (flat config format). Three configuration layers:

```
// eslint.config.mjs — Structure
export default [
```

```
// 1) Global ignores
{ ignores: [".next/**", "scripts/**", "prisma/**", ...] },

// 2) Base JS rules
js.configs.recommended,

// 3) Next.js rules (pages, components, lib, app)
{ files: ["pages/**", "components/**", "lib/**", "app/**"],
  plugins: { "@next/next": next },
  rules: { ...next.configs.recommended.rules,
    ...next.configs["core-web-vitals"].rules } },

// 4) TypeScript rules (all .ts/.tsx)
...tseslint.config({
  files: ["**/*.ts,tsx"],
  languageOptions: { parserOptions: { projectService: true } },
  rules: {
    "@typescript-eslint/no-unused-vars": "off",
    "@typescript-eslint/no-explicit-any": "off"
  }
}),

// 5) Node scripts (currently ignored, available for future)
{ files: ["scripts/**", "tools/**", "netlify/**"],
  languageOptions: { globals: globals.node, sourceType: "module" },
  rules: { "no-console": "off" } }
];
```

20.2 Critical Rule: Restricted Imports

⚠ WARNING

Client components (`"use client"` files) must NEVER import:

- `@prisma/client` — Server-only database access
- Any file from `lib/server/` — Server-only logic
- `fs` , `path` , `child_process` — Node.js built-ins

The `check:content-boundary` script enforces this at CI time. Violation = build failure.

```
pnpm check:content-boundary # Scans for server imports in client code
```

20.3 Override Contexts

Context	Files	Special Rules
UI Files	components/**, app/**/page.tsx	React rules, no-console warn
Server Files	app/api/**, lib/server/**	Node globals, console allowed
Lib/Scripts	lib/**, scripts/**	Relaxed unused-vars, console allowed

20.4 Prettier Configuration

```
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": false,
  "printWidth": 80,
  "tabWidth": 2,
  "useTabs": false,
  "endOfLine": "lf"
}
```

KEY PRINCIPLE

Double quotes are the institutional standard. Single quotes are prohibited in committed code. The `endOfLine: "lf"` setting prevents Windows CRLF contamination — all files must be LF-only regardless of developer OS.

Command	Action
<code>pnpm format</code>	Auto-format all files
<code>pnpm format:check</code>	Check without modifying (CI mode)

20.5 TypeScript Strict Mode

TypeScript is configured with strict mode enabled. The `typecheck` command runs `tsc --noEmit` and must pass before any merge to main.

```
pnpm typecheck      # Full strict type check
pnpm typecheck:safe # Non-blocking (logs errors, exits 0)
```

20.6 Pre-commit Hooks (Husky)

Husky runs on every `git commit`:

```
# .husky/pre-commit
pnpm mdx:integrity
pnpm mdx:gate
```

mdx:integrity — Verifies all MDX files have valid component references, no broken imports, no malformed frontmatter.

mdx:gate — Blocks commit if any MDX file contains illegal JSX patterns (raw HTML tags that should be components, unsafe string interpolation).

⚠ PROHIBITION

Never bypass pre-commit hooks with `--no-verify`. If the hooks fail, the MDX content is broken and **MUST** be fixed before commit. Use `pnpm fix:mdx` to auto-repair common issues.

20.7 The `validate` Command

```
pnpm validate # typecheck && lint && test
```

This is the single command that proves code is ready for review. It must pass locally before opening a PR. CI runs it automatically.

20.8 Code Review Standards

Every PR must satisfy:

- 1. `pnpm validate` **passes** — Types, lint, and tests green
- 2. **No new any types** without justification comment
- 3. **All API routes have Zod validation** — No unvalidated input
- 4. **Security-sensitive changes reviewed by Founder** — Auth, encryption, access control
- 5. **MDX changes pass integrity gate** — Content is code; treat it accordingly
- 6. **No secrets in diff** — Environment variables only via platform config

Chapter 21: Performance Standards

21.1 Core Web Vitals Targets

Metric	Target	Measurement
LCP (Largest Contentful Paint)	< 2.5s	75th percentile
FID (First Input Delay)	< 100ms	75th percentile
CLS (Cumulative Layout Shift)	< 0.1	75th percentile
TTFB (Time to First Byte)	< 800ms	Server response
INP (Interaction to Next Paint)	< 200ms	All interactions

———— **KEY PRINCIPLE**

Performance is a feature. A slow institutional platform signals incompetence. Every page must meet Core Web Vitals "Good" thresholds. No exceptions for "complex" pages — refactor until fast.

21.2 ISR Strategy (Incremental Static Regeneration)

Page Type	Revalidation Period	Rationale
Blog/editorial	3600s (1 hour)	Content updates infrequently
Lexicon entries	3600s	Static reference material
Vault pages	3600s	Gated content, rarely changes
Dashboard	0 (dynamic)	Real-time data required
Landing pages	3600s	Marketing content
API routes	N/A (always dynamic)	Server-side only

Manual revalidation available via `revalidatePath()` for critical content updates that cannot wait.

21.3 Bundle Optimization

Standalone Output — Next.js standalone mode (`output: "standalone"` in `next.config.mjs`) produces a self-contained deployment artifact. No `node_modules` folder in production.

Tree-shaking — All imports must be specific. Never `import * as X from 'library'`.

Dynamic Imports — Heavy components loaded on demand:

```
const HeavyChart = dynamic(() => import('@components/charts/HeavyChart'), {
  loading: () => <ChartSkeleton />,
  ssr: false,
});
```

Bundle Analysis:

```
pnpm build:analyze # cross-env ANALYZE=true pnpm build
```

Opens webpack-bundle-analyzer in browser. Use to identify oversized chunks.

21.4 Image Optimization

- **Sharp** — Server-side image processing (resize, format conversion)
- **next/image** — Automatic responsive images, WebP/AVIF conversion, lazy loading
- **Assets pipeline** — `pnpm assets:optimize` and `pnpm assets:enterprise` for bulk processing

```
pnpm assets:optimize          # Standard optimization
pnpm assets:enterprise        # Premium quality pipeline
pnpm assets:enterprise:production # Production-grade with AVIF
pnpm optimize:premium         # Ultra-quality images only
```

21.5 Redis Caching Layer (Optional)

Redis provides an optional caching layer via Upstash. The system degrades gracefully when Redis is unavailable.

```
REDIS_URL=redis://127.0.0.1:6379
REDIS_DISABLED=true           # Set to false to enable
USE_REDIS=false               # Feature flag
UPSTASH_REDIS_REST_URL=       # Production Upstash endpoint
UPSTASH_REDIS_REST_TOKEN=     # Production Upstash token
```

— KEY PRINCIPLE

Redis is a performance enhancement, not a requirement. Every feature must work without Redis. The `REDIS_DISABLED=true` flag causes all cache operations to no-op silently. Never make Redis a hard dependency.

21.6 Critical CSS Inlining (Critters)

Critters extracts and inlines above-the-fold CSS during build, eliminating render-blocking stylesheets. Configured automatically via Next.js optimization pipeline.

21.7 Monitoring: Vercel Speed Insights

Real User Monitoring (RUM) via `@next/third-parties`:

- Page load performance (real users)
- Core Web Vitals (field data)
- Geographic performance distribution

- Device-specific bottlenecks

Enabled via `ENABLE_ANALYTICS=true` feature flag.

PART VIII — OPERATIONS & DEPLOYMENT

Chapter 22: Build Pipeline

22.1 Build Command Breakdown

The production build command is `pnpm build:netlify`. It executes four stages in sequence:

```
build:netlify = mdx:gate → contentlayer:clean → mdx:integrity → build:fast
```

Stage	Script	Purpose
1	<code>pnpm mdx:gate</code>	Block build if illegal JSX detected in MDX
2	<code>pnpm contentlayer:clean</code>	Clear stale Contentlayer cache
3	<code>pnpm mdx:integrity</code>	Verify all MDX component references resolve
4	<code>pnpm build:fast</code>	PDF registry + Next.js build + standalone cleanup

The `build:fast` stage itself runs:

```
build:fast = tsx scripts/build-pdf-registry-json.ts
             → cross-env NODE_OPTIONS="--max-old-space-size=7168" next build --webpack
             → node scripts/clean-standalone.mjs
```

22.2 Node Configuration

```
NODE_OPTIONS="--max-old-space-size=7168
```

The 7168 MB (7 GB) heap allocation is required because:

- Contentlayer processes 100+ MDX files with complex frontmatter
- PDF registry generation holds all asset metadata in memory
- Webpack builds the full application graph including all dynamic routes

⚠ WARNING

Do not reduce this value below 7168 MB. Build will OOM on the Contentlayer + Next.js combined phase. The Netlify build environment provides 8 GB — this leaves ~800 MB headroom for OS and tools.

22.3 Standalone Output Mode

`output: "standalone"` in `next.config.mjs` produces a self-contained server. The `clean-standalone.mjs` script removes unnecessary files from the standalone output to reduce deployment size.

Output structure:

```

.next/
├─ standalone/      # Self-contained Node.js server
│  └─ server.js     # Entry point
│  └─ node_modules/ # Minimal dependencies
│  └─ ...
├─ static/          # Static assets (CSS, JS chunks)
├─ server/          # Server-side code
└─ cache/           # ISR cache

```

22.4 Content Compilation

Contentlayer2 compiles MDX content into typed JSON during build:

```

pnpm contentlayer:build # Standard build
pnpm contentlayer:safe  # Build with cache clear
pnpm contentlayer:clean # Remove .contentlayer directory

```

The compiled content lives in `.contentlayer/generated/` and is consumed by Next.js at build time.

22.5 PDF Registry Generation

During build, `scripts/build-pdf-registry-json.ts` scans all content and generates `lib/pdf/pdf-registry.generated.ts`. This registry maps slugs to PDF assets, validates existence, and provides type-safe access.

```

pnpm pdf:registry:build # Generate registry
pnpm pdf:registry:verify # Validate registry integrity

```

22.6 Build Failure Troubleshooting

Symptom	Cause	Fix
MDX gate failed	Illegal JSX in MDX files	<code>pnpm fix:mdx</code> then review
MDX integrity failed	Broken component references	Check MDX imports match components/
JavaScript heap out of memory	NODE_OPTIONS not set	Ensure <code>--max-old-space-size=7168</code>
Contentlayer error	Stale cache	<code>pnpm contentlayer:clean</code> then rebuild
Module not found	Import path error	<code>pnpm fix:imports</code>
Type error	TypeScript strict violation	<code>pnpm typecheck</code> to identify
PDF registry invalid	Missing PDF files	<code>pnpm pdf:registry:build</code>
Prisma client not generated	Missing generate step	<code>pnpm db:generate</code>

Chapter 23: Deployment

23.1 Netlify Configuration (`netlify.toml`)

```

[build]
  publish = ".next"

```

```

command = "pnpm build:netlify"

[build.environment]
  NODE_VERSION = "20.20.0"
  NODE_OPTIONS = "--max-old-space-size=7168"
  CI = "true"
  NEXT_TELEMETRY_DISABLED = "1"
  NEXT_DISABLE_SOURCEMAPS = "true"
  CONTENTLAYER_DISABLE_WARNINGS = "1"
  NETLIFY_USE_PNPM = "true"
  Pnpm_FLAGS = "--frozen-lockfile"
  Pnpm_VERSION = "10.29.3"

[[plugins]]
  package = "@netlify/plugin-nextjs"

[[plugins]]
  package = "@netlify/plugin-emails"

[functions]
  directory = "netlify/functions_src/functions"
  node_bundler = "nft"
  external_node_modules = ["@prisma/client", ".prisma", "prisma"]

```

23.2 Environment Setup

Context	NEXT_PUBLIC_APP_ENV	NEXT_PUBLIC_SITE_URL
Production	production	https://www.abrahamoflondon.org
Deploy Preview	staging	Auto-generated Netlify URL
Local	development	http://localhost:3000

23.3 Headers and Security

All responses include security headers (configured in `next.config.mjs` and/or Netlify headers):

Header	Value	Purpose
X-Content-Type-Options	nosniff	Prevent MIME sniffing
X-Frame-Options	DENY	Prevent clickjacking
Referrer-Policy	strict-origin-when-cross-origin	Limit referrer leakage
Strict-Transport-Security	max-age=31536000; includeSubDomains	Enforce HTTPS (1 year)
X-XSS-Protection	1; mode=block	Legacy XSS filter
Permissions-Policy	camera=(), microphone=(), geolocation=()	Restrict browser APIs

23.4 Redirects

Over 100 redirects configured in `netlify.toml` :

Category 1 — Domain canonicalization:

```
[[redirects]]
  from = "https://abrahamoflondon.org/*"
  to = "https://www.abrahamoflondon.org/:splat"
  status = 301
  force = true
```

Category 2 — Legacy route rewrites:

```
[[redirects]]
  from = "/essays/*"
  to = "/blog/:splat"
  status = 301

[[redirects]]
  from = "/insights/:slug"
  to = "/blog/:slug"
  status = 301

[[redirects]]
  from = "/downloads/vault/*"
  to = "/vault/:splat"
  status = 301
```

Category 3 — PDF canonical aliases (100+):

```
[[redirects]]
  from = "/lexicon/brotherhood.pdf"
  to = "/assets/downloads/brotherhood.pdf"
  status = 301
  force = true
```

These ensure every PDF has exactly one canonical URL. Legacy direct-file paths redirect to the canonical asset location.

23.5 Scheduled Functions

```
[[schedule]]
  path = "/api/cleanup-download-tokens"
  schedule = "0 2 * * *" # Daily at 2 AM UTC
```

Scheduled functions handle:

- Download token cleanup (expired tokens purged daily)
- Stale session removal
- Audit log rotation

23.6 Deploy Hooks

Netlify deploy hooks trigger builds from external events:

- Content CMS publish → webhook → rebuild
- Manual trigger via Netlify dashboard
- `pnpm deploy` — validates links, commits, pushes (auto-deploys on push to main)

23.7 Rollback Procedure

1. Open Netlify dashboard → Deploys
2. Find last known-good deploy
3. Click "Publish deploy" on that entry
4. Verify site is restored
5. Investigate and fix the broken commit on a branch
6. Re-deploy when fixed

———— **KEY PRINCIPLE**

Netlify retains all previous deploys indefinitely. Rollback is instant — no rebuild required. Always roll back first, investigate second.

23.8 Docker Deployment (Alternative)

For PDF generation and local full-stack development:

```
FROM node:20-slim

# Install LibreOffice + Chromium for PDF generation
RUN apt-get update && apt-get install -y \
    libreoffice chromium fonts-liberation \
    libnss3 libatk-bridge2.0-0 libxcomposite1 \
    --no-install-recommends

ENV PUPPETEER_SKIP_CHROMIUM_DOWNLOAD=true
ENV PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium

WORKDIR /app
COPY package.json pnpm-lock.yaml* ./
RUN npm install --legacy-peer-deps --ignore-scripts
COPY . .
RUN npm install -g tsx

CMD ["tsx", "scripts/pdf/unified-pdf-generator.ts",
    "--scan-content", "--overwrite", "--strict"]
```

23.9 Docker Compose

```
pnpm docker:build    # Build image
pnpm docker:run      # Run container (port 3000)
pnpm docker:compose  # Full stack via docker-compose
```

Chapter 24: Environment Management

24.1 File Structure

File	Purpose	Git-tracked?
<code>.env.example</code>	Template with all variables documented	Yes
<code>.env</code>	Base environment (non-sensitive defaults)	No
<code>.env.local</code>	Local overrides with real secrets	No

⚠ PROHIBITION

NEVER commit `.env` or `.env.local` to Git. These files contain secrets. Only `.env.example` is tracked — it contains placeholder values (`CHANGE_ME`) and documentation comments.

24.2 Variable Categories (15 Categories)

#	Category	Count	Critical?
1	Application	7	Yes
2	Database	2	Yes
3	Redis	5	No (optional)
4	Authentication	8	Yes
5	Admin	8	Yes
6	Cron/Internal	3	Yes
7	Brand/Identity	8	Yes
8	Email	7	Partial
9	Inner Circle	4	Yes
10	Enterprise/Diagnostics	5	Yes
11	Sovereign/OGR	3	Yes
12	Payments (Stripe)	2	No (optional)
13	AI	1	No (optional)
14	OAuth Integrations	5	No (optional)
15	PDF	4	Yes
16	Security	2	Yes
17	Feature Flags	6	No (defaults)
18	Development	3	No (dev only)

24.3 Required vs Optional Variables

Required for build:

- `DATABASE_URL` (or `SKIP_DB=true`)

- NEXTAUTH_SECRET
- NEXT_PUBLIC_SITE_URL

Required for runtime:

- All Authentication variables
- All Admin variables
- DATABASE_URL + DIRECT_URL
- Email provider credentials (if notifications enabled)

Optional (graceful degradation):

- REDIS_URL — Caching disabled if missing
- STRIPE_SECRET_KEY — Payment features hidden
- OPENAI_API_KEY — AI features disabled
- GOOGLE_OAUTH_CLIENT_ID — Calendar sync unavailable

24.4 Secrets Management

———— **KEY PRINCIPLE**

Secrets live in the deployment platform (Netlify environment variables), never in code. The only local file containing secrets is `.env.local` which is git-ignored. If a secret appears in a git diff, rotate it immediately.

Rotation procedure:

1. Generate new secret value
2. Update in Netlify environment variables
3. Trigger redeploy
4. Revoke old secret value
5. Log rotation in governance audit

24.5 Local Development Setup

```

cp .env.example .env.local
# Edit .env.local — fill in CHANGE_ME values

```

Minimum viable `.env.local` for local development:

```

DATABASE_URL="file:./prisma/dev.db"
DIRECT_URL="file:./prisma/dev.db"
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=local-dev-secret-min-32-chars-long
JWT_SECRET=local-dev-jwt-secret-32-chars
SKIP_DB=false
REDIS_DISABLED=true

```

24.6 Feature Flags

Flag	Default	Effect When true
ENABLE_ANALYTICS	false	Activates Vercel Analytics + Speed Insights
ENABLE_PDF_GENERATION	true	Allows runtime PDF creation

ENABLE_EMAIL_NOTIFICATIONS	false	Enables Resend email delivery
SKIP_DB	false	Bypasses all database operations
REDIS_DISABLED	true	Disables Redis cache layer
DEBUG_CONTENTLAYER	false	Verbose Contentlayer logging

24.7 Graceful Degradation

The platform is designed to function with minimal services:

Service Missing	Behavior
Redis	Cache operations no-op, all data fetched fresh
Stripe	Payment UI hidden, checkout disabled
OpenAI	AI features return fallback/static responses
Email (Resend)	Notifications queued but not sent
Google OAuth	Calendar integration unavailable
Neon (production DB)	Falls back to SQLite if configured

```
pnpm env:check      # Validates all required vars are set
pnpm env:validate    # Full validation with type checking
```

Chapter 25: Monitoring & Observability

25.1 Vercel Analytics

Web Vitals tracking for all pages. Captures:

- LCP, FID, CLS, TTFB, INP
- Page-level performance breakdown
- Geographic distribution
- Device/browser segmentation

Enabled by `ENABLE_ANALYTICS=true` and the `@next/third-parties` integration.

25.2 Vercel Speed Insights

Real-user monitoring (RUM). Provides field data vs synthetic lab data. Tracks actual user experience across:

- Network conditions
- Device capabilities
- Geographic latency

25.3 Custom Telemetry

Two internal telemetry endpoints:

Global Telemetry:

```
POST /api/telemetry/global
Body: { event, properties, timestamp }
```

Captures: page views, navigation patterns, session duration, feature usage.

Resonance Telemetry:

```
POST /api/telemetry/resonance
Body: { contentSlug, engagementType, duration, depth }
```

Captures: content engagement, scroll depth, time-on-page, interaction density.

25.4 Health Checks

Netlify Function Health:

```
GET /api/cleanup-download-tokens (scheduled)
```

Application Health:

```
GET /api/v2/health
Response: { status: "ok", timestamp, version, services: {...} }
```

System Health Script:

```
pnpm health # tsx scripts/system/check-system-health.ts
```

Checks: database connectivity, Redis (if enabled), disk space, memory usage, external service availability.

25.5 Audit Logging

Three audit models capture all security-relevant events:

Model	Purpose	Fields
GovernanceLog	Administrative actions	action, performedBy, target, details
SecurityLog	Auth/access events	event type, IP, userId, metadata
SystemAuditLog	System-wide audit trail	actor, action, resource, severity, metadata

All audit entries are append-only. Deletion requires Founder authorization.

```
// Standard audit log entry
await prisma.systemAuditLog.create({
  data: {
    actorType: 'USER',
    actorId: userId,
    action: 'DOWNLOAD_ARTIFACT',
    resourceType: 'pdf',
    resourceId: artifactId,
    severity: 'info',
    metadata: JSON.stringify({ ip, userAgent }),
  },
})
```

```
}  
});
```

25.6 Error Tracking Architecture

The platform is Sentry-ready but does not currently ship with Sentry enabled. Error boundaries exist at:

- App-level (`app/error.tsx`)
- Page-level (per-route error boundaries)
- API routes (try/catch with audit logging)

All unhandled errors are:

1. Caught by error boundary
2. Logged to `SystemAuditLog` with severity `error`
3. User shown generic error page (no stack traces in production)

25.7 Log Levels

```
LOG_LEVEL=warn    # debug | info | warn | error | critical
```

Level	When Used
debug	Verbose development tracing
info	Normal operations (startup, requests)
warn	Recoverable issues (cache miss, slow query)
error	Operation failed, user impacted
critical	System integrity compromised

PART IX — SECURITY

Chapter 26: Security Architecture

26.1 Defense in Depth

NETWORK LAYER
- HTTPS enforced (HSTS 1yr)
- CORS whitelist
- Rate limiting (100 req/15 min)
APPLICATION LAYER
- CSRF protection (synchronizer token)
- Input validation (Zod schemas on ALL routes)
- Output encoding (React default + DOMPurify)
- Content Security Policy headers
DATA LAYER

- Access tier enforcement (8 levels)
- Entitlement system (TIER, PRODUCT, ARTIFACT)
- Encryption at rest (Neon TLS, AES-256-GCM for tokens)

AUDIT LAYER

- All access attempts logged
- GovernanceLog, SecurityLog, SystemAuditLog
- Append-only (no deletion without Founder auth)

26.2 Rate Limiting

Implementation: Upstash rate limiter (production) + in-memory fallback (development).

```
RATE_LIMIT_MAX_REQUESTS=100
RATE_LIMIT_WINDOW_MS=900000 # 15 minutes
```

Endpoint Type	Limit	Window
Public API	100 requests	15 minutes
Auth endpoints	10 requests	5 minutes
Admin API	1000 requests	15 minutes
Webhook receivers	Unlimited	N/A

When rate limit is exceeded:

- Response: 429 Too Many Requests
- Retry-After header included
- Event logged to RateLimitLog model

26.3 CORS

Whitelist-based CORS. Only approved origins may make cross-origin requests:

```
ALLOWED_ORIGINS=http://localhost:3000,https://www.abrahamoflondon.org
```

All other origins receive no CORS headers → browser blocks the request.

26.4 CSRF Protection

Synchronizer token pattern:

1. Server generates CSRF token on page load
2. Token embedded in form/stored in meta tag
3. All state-changing requests must include token
4. Server validates token matches session

```
CSRF_SECRET=<random-32-char-secret>
```

26.5 Input Validation

KEY PRINCIPLE

Every API route validates input with Zod. No raw `request.body` access without schema validation. This is non-negotiable. Unvalidated input is the #1 source of security vulnerabilities.

```
// Standard pattern for all API routes
import { z } from 'zod';

const schema = z.object({
  email: z.string().email(),
  name: z.string().min(1).max(200),
  tier: z.enum(['member', 'inner_circle', 'client']),
});

export async function POST(request: Request) {
  const body = await request.json();
  const parsed = schema.safeParse(body);

  if (!parsed.success) {
    return Response.json(
      { ok: false, error: 'Validation failed', code: 'INVALID_INPUT' },
      { status: 400 }
    );
  }

  // Use parsed.data - fully typed and validated
}
```

26.6 Output Encoding

- **React default** — All JSX expressions are auto-escaped
- **DOMPurify** — Used for any user-generated HTML that must be rendered raw
- **Never use** `dangerouslySetInnerHTML` without DOMPurify sanitization

Chapter 27: Authentication Security

27.1 JWT Configuration

Parameter	Value
Algorithm	HS256
Expiry	30 days
Secret	NEXTAUTH_SECRET (min 32 chars)
Issuer	Abraham of London

```
JWT_ALGORITHM=HS256
JWT_EXPIRES_IN=30d
NEXTAUTH_SECRET=<random-64-char-secret>
JWT_SECRET=<random-64-char-secret>
```

27.2 Cookie Security

Attribute	Value	Purpose
HttpOnly	true	Prevents JavaScript access
Secure	true (production)	HTTPS only
SameSite	Strict	Prevents CSRF via cookies
Prefix	ao1	Namespace isolation
Path	/	Site-wide

```
SESSION_COOKIE_PREFIX=ao1
ACCESS_COOKIE_SECRET=<random-32-char-secret>
```

27.3 OAuth Token Encryption

OAuth tokens (Google Calendar, Slack) are encrypted at rest using AES-256-GCM:

```
OAUTH_TOKEN_ENCRYPTION_KEY=<min-32-char-secret>
```

Encryption flow:

- 1. OAuth callback receives access/refresh tokens
- 2. Tokens encrypted with AES-256-GCM before database storage
- 3. Decrypted only at point of use (API call to provider)
- 4. IV is unique per encryption operation

27.4 Password Storage

Method	Use Case
Argon2	All user passwords
bcrypt	Legacy (migration path)
Plaintext	NEVER (except ADMIN_USER_PASSWORD env var for bootstrap)

⚠ WARNING

The ADMIN_USER_PASSWORD environment variable exists solely for initial system bootstrap. Once the admin account is created, authentication switches to Argon2-hashed passwords. The plaintext env var should be rotated/removed after first login.

27.5 MFA Implementation

TOTP-based MFA via @otplib/preset-default :

Parameter	Value
Algorithm	SHA1 (TOTP standard)
Digits	6

Period	30 seconds
Window	1 (allows ± 1 period drift)

```
// MFA setup stored in MfaSetup model
model MfaSetup {
  id      String    @id @default(cuid())
  memberId String
  method  MfaMethod @default(totp)
  secret  String    // Encrypted TOTP secret
  verified Boolean   @default(false)
  createdAt DateTime @default(now())
}
```

27.6 Session Management

- Server-side session validation on every request
- Token rotation on privilege escalation
- Automatic expiry after `AOL_SESSION_TTL_DAYS` (default: 30)
- Revocation logged to `SecurityLog`

```
// Session model
model Session {
  id      String    @id @default(cuid())
  memberId String
  token   String    @unique
  status  SessionStatus @default(active)
  expiresAt DateTime
  createdAt DateTime @default(now())
}
```

27.7 Admin Bootstrap

Initial admin creation uses environment whitelist:

```
ADMIN_ALLOWED_EMAILS=admin@abrahamoflondon.org
ADMIN_USER_EMAIL=admin@abrahamoflondon.org
```

Only emails in `ADMIN_ALLOWED_EMAILS` can be granted `ADMIN` or `OWNER` roles. This is a compile-time constraint — not a runtime check that can be bypassed.

Chapter 28: Data Protection

28.1 Access Tier Enforcement

Eight access levels, strictly hierarchical:

Level	Enum Value	Who
0	public	Anyone

1	member	Registered users
2	inner_circle	Inner Circle members
3	restricted	Specific entitlement holders
4	client	Active clients
5	legacy	Legacy program members
6	architect	System architects
7	owner	Founder only
8	top_secret	Founder + explicit grant

Every content piece, API route, and artifact has an assigned `AccessTier` . The enforcement middleware checks:

1. User's highest entitlement tier
2. Resource's required tier
3. Grant-based overrides (specific artifact entitlements)

28.2 Entitlement System

Three entitlement types govern access:

Type	Purpose	Example
TIER	Access level grant	inner_circle tier access
PRODUCT	Product-specific access	Diagnostic suite access
ARTIFACT	Individual artifact access	Specific PDF download

```
model Entitlement {
  id          String          @id @default(cuid())
  userId      String
  type        EntitlementType // TIER | PRODUCT | ARTIFACT
  key         String          // The specific grant
  status      EntitlementStatus // ACTIVE | REVOKED | EXPIRED
  issuedAt    DateTime
  expiresAt   DateTime?
  issuedBy    String?
  revokedBy   String?
  reason      String?
}
```

28.3 Audit Trail

All access attempts are logged regardless of success/failure:

```
model AccessAuditLog {
  id          String          @id @default(cuid())
  userId      String?
```

```

resourceType String
resourceId   String
accessType   AccessType // VIEW | DOWNLOAD | METADATA
granted      Boolean
reason       String?
ip           String?
userAgent    String?
createdAt    DateTime @default(now())
}

```

———— **KEY PRINCIPLE**

Audit logs are evidence. They prove who accessed what, when, and whether access was granted. They are append-only, indexed by user and resource, and retained indefinitely. No deletion without written Founder authorization.

28.4 Encryption at Rest

Data	Encryption
Database (Neon)	TLS in transit, encrypted storage
OAuth tokens	AES-256-GCM (application-level)
PDF watermarks	HMAC-SHA256 (integrity)
Session tokens	Random + hashed storage
MFA secrets	Encrypted column

28.5 Encryption in Transit

- All connections via HTTPS (enforced by HSTS)
- Database connections via TLS (Neon serverless driver)
- Redis connections via TLS (Upstash)
- No plaintext HTTP allowed in production

28.6 Anonymization

Two salt values support data anonymization:

```

AOL_HASH_SALT=<random-secret>      # General hashing
ANONYMITY_SALT=<random-secret>      # User identity anonymization
DENYLIST_PEPPER=<random-secret>    # Denylist matching without storing emails

```

Used for:

- Analytics aggregation (no PII in telemetry)
- Denylist matching (hash comparison, never store raw)
- Report anonymization (team assessments without names)

28.7 Data Retention Policies

Data Type	Retention	Justification
Audit logs	Indefinite	Legal/compliance requirement

Session data	90 days after expiry	Cleanup via scheduled function
Download tokens	72 hours	Purged daily at 2 AM UTC
Telemetry events	365 days	Annual performance review
User data	Until deletion request	GDPR compliance
Assessment data	5 years	Client contract requirement

PART X — DEVELOPER OPERATIONS

Chapter 29: Local Development Setup

29.1 Step-by-Step Setup

Prerequisites:

- Node.js 20+ (exact: 20.20.0 for parity with CI)
- pnpm 10.33+ (install: `npm install -g pnpm@latest`)
- Git 2.40+
- (Optional) LibreOffice — for PDF generation
- (Optional) Docker — for containerized PDF pipeline

Step 1 — Clone:

```
git clone https://github.com/AbrahamofLondon/aol-check-visual.git
cd aol-check-visual
```

Step 2 — Install dependencies:

```
pnpm install
```

Step 3 — Environment setup:

```
cp .env.example .env.local
# Edit .env.local — fill all CHANGE_ME values
# Minimum: DATABASE_URL, NEXTAUTH_SECRET, JWT_SECRET
```

Step 4 — Database setup:

```
# Option A: SQLite (simplest for local dev)
pnpm db:init # setup + migrate + seed

# Option B: PostgreSQL via Prisma
pnpm db:generate && pnpm db:push && pnpm db:seed
```

Step 5 — Start development:

```
pnpm dev
```

Step 6 — Verify:

Open `http://localhost:3000`

29.2 Windows-Specific Notes

```
IS_WINDOWS=true # Set in .env.local on Windows
```

Issue	Solution
Path separators	Scripts normalize <code>\</code> → <code>/</code> when <code>IS_WINDOWS=true</code>
LibreOffice path	<code>C:\Program Files\LibreOffice\program\soffice.com</code>
Line endings	Git: <code>core.autocrlf=input</code> . Prettier enforces LF.
Long paths	Enable: <code>git config --system core.longpaths true</code>
Encoding	<code>pnpm fix:encoding</code> resolves UTF-8 BOM issues

```
pnpm dev:windows # cross-env IS_WINDOWS=true contentlayer2 build && next dev
pnpm fix:windows # Auto-fix Windows path issues
```

29.3 Common Setup Issues

Problem	Fix
<code>prisma generate</code> fails	<code>pnpm fix:prisma</code> (re-runs generate)
Contentlayer cache corrupt	<code>pnpm contentlayer:clean && pnpm contentlayer:build</code>
Port 3000 in use	Kill process or use <code>pnpm preview</code> (port 3001)
Node version mismatch	Use nvm: <code>nvm use 20</code>
pnpm lockfile conflict	<code>pnpm install --frozen-lockfile</code> Or <code>pnpm reinstall</code>

Chapter 30: Development Workflow

30.1 Branch Strategy

`main` (production) ← feature branches

- **main** — Protected. Auto-deploys to Netlify production.
- **feature/*** — All development work. Named descriptively.
- No `develop` branch. No staging branch. Ship directly.

30.2 Commit Conventions

All commits use a prefix convention:

Prefix	Use
Feature:	New functionality
Fix:	Bug fix
Refactor:	Code restructure (no behavior change)
Content:	MDX/content changes
Style:	CSS/visual changes
Docs:	Documentation updates
Test:	Test additions/changes
Build:	Build system changes
Deploy:	Deployment configuration
Security:	Security fixes/hardening

Example:

```
Feature: public /lexicon/[slug] route and index page
Fix: PDF registry validation for empty slugs
Refactor: extract audit logging to shared utility
```

30.3 Pre-commit Validation

Every commit triggers Husky hooks:

```
git commit → .husky/pre-commit → mdx:integrity + mdx:gate
```

If either check fails:

1. Commit is blocked
2. Error output shows which files failed
3. Run `pnpm fix:mdx` for auto-repair
4. Or manually fix the MDX issue
5. Re-stage and commit again

30.4 PR Requirements

1. Branch is up-to-date with main
2. `pnpm validate` passes (types + lint + tests)
3. No merge conflicts
4. Descriptive title with commit prefix
5. Security-sensitive changes flagged for Founder review

30.5 Code Review Checklist

- ☐ Types correct (no new `any` without comment)
- ☐ Zod validation on all API inputs
- ☐ Error handling follows `{ok, error, data, code}` pattern
- ☐ No secrets in code

- ☐ Access tier enforcement for new routes
- ☐ Audit logging for security-relevant operations
- ☐ Tests for domain logic changes
- ☐ MDX integrity passes

30.6 Deploy-on-Merge

Merging to `main` triggers automatic Netlify deployment:

1. Merge PR → push to main
2. Netlify webhook fires
3. `pnpm build:netlify` runs
4. Site deployed to production (typically 2-4 minutes)
5. Verify via `pnpm verify:production`

Chapter 31: Scripts Reference

31.1 Build & Development

Script	Command
<code>dev</code>	MDX gate → PDF registry → Contentlayer build → Next.js dev
<code>dev:full</code>	Env check → concurrent Contentlayer watch + Next.js dev
<code>dev:windows</code>	Windows-compatible dev server
<code>dev:profile</code>	Dev with Next.js profiling enabled
<code>start</code>	Production server (after build)
<code>preview</code>	Production server on port 3001
<code>build</code>	Generate EPUBs → Next.js build
<code>build:netlify</code>	MDX gate → CL clean → MDX integrity → build:fast
<code>build:fast</code>	PDF registry → Next.js build (7GB heap) → clean standalone
<code>build:safe</code>	Vault fix → encoding fix → validate → build
<code>build:ci</code>	prebuild:ci → build
<code>build:analyze</code>	Build with webpack-bundle-analyzer
<code>build:verbose</code>	Build with DEBUG=*
<code>clean</code>	Remove .next, .contentlayer, cache
<code>clean:hard</code>	Deep clean (scripts/clean-project.js)
<code>clean:all</code>	clean + assets:clean + content:clean + module cache

31.2 Content

Script	Command
--------	---------

mdx:gate	Block if illegal JSX in MDX
mdx:integrity	Verify all MDX component references
mdx:sanitize	Sanitize all MDX files
contentlayer:build	Compile MDX → JSON
contentlayer:watch	Watch mode compilation
contentlayer:safe	Build with cache clear
contentlayer:clean	Remove .contentlayer directory
content:normalize	Fix whitespace in content files
content:validate	Validate frontmatter schemas
content:clean	Remove content build artifacts

31.3 PDF

Script	Command
pdf:build:full	Registry full → generate all → sync
pdf:generate-all	Unified generator (Puppeteer + LibreOffice)
pdf:registry:build	Generate pdf-registry.generated.ts
pdf:registry:full	Build + registry + variants
pdf:registry:audit	Audit registry consistency
pdf:registry:verify	Verify registry file exists and valid
pdf:sync	Scan content for PDF references
pdf:audit	Full audit (registry + links + dupes + canonical + governance)
pdf:verify-links	Check hardcoded PDF link validity
pdf:duplicates	Report duplicate PDF files
pdf:canonical	Generate canonical path decisions
pdf:redirect-candidates	Find URLs needing redirects
pdf:governance	Verify PDF governance rules
pdf:rewrite-links	Rewrite to download page URLs
pdf:materialize	Create canonical redirect files
pdf:enforce	Enforce governance (fail on violation)
pdf:repair	Auto-repair empty/corrupt PDFs
pdf:list	List all PDF assets

pdf:validate	Validate PDF file integrity
pdf:stats	Generate PDF statistics
pdf:ebook	Render EPUB format
pdf:vault-pack	Build vault download pack
pdf:ops	PDF operations (strict, sequential)

31.4 Database

Script	Command
db:generate	Generate Prisma client
db:push	Push schema to database
db:migrate	Create migration (dev)
db:deploy	Apply migrations (production)
db:seed	Seed database with initial data
db:studio	Open Prisma Studio GUI
db:reset	Reset + re-seed (destructive)
db:status	Check migration status
db:init	SQLite: setup + migrate + seed
db:refresh	SQLite: reset + migrate + seed
db:sync:all	Sync both Prisma + SQLite
db:backup	Backup database
db:restore	Restore from backup
db:validate	Validate data integrity
db:stats	Database statistics

31.5 Vault

Script	Command
vault:sync	Master vault synchronization (8GB heap)
vault:fix	Repair vault issues
vault:audit	Audit vault integrity
vault:check	Build manifest + validate links
vault:manifest	Generate vault manifest

<code>vault:sign</code>	Sign vault contents
<code>vault:ready</code>	CL safe → audit → glossary → check → manifest
<code>vault:dry</code>	Dry-run repair (no changes)
<code>vault:generate</code>	Batch content generation
<code>vault:deploy</code>	Audit → sync → build
<code>vault:glossary</code>	Inject glossary terms

31.6 Testing

Script	Command
<code>test</code>	Vitest (default: watch mode)
<code>test:ui</code>	Vitest with browser UI
<code>test:watch</code>	Watch specific test files
<code>test:coverage</code>	Run with V8 coverage
<code>test:coverage:html</code>	Coverage with HTML report
<code>test:all</code>	test + check:all + pdf:audit
<code>test:integration</code>	Integration tests (sequential)
<code>test:performance</code>	Performance tests (sequential)
<code>test:deal-fusion</code>	Deal fusion domain tests
<code>test:predictive</code>	Predictive intelligence tests
<code>test:decision</code>	Decision engine tests
<code>validate</code>	typecheck + lint + test

31.7 Quality

Script	Command
<code>lint</code>	Next.js ESLint
<code>lint:fix</code>	ESLint with auto-fix
<code>lint:safe</code>	Lint (non-blocking)
<code>format</code>	Prettier write all files
<code>format:check</code>	Prettier check (CI mode)
<code>typecheck</code>	tsc --noEmit (strict)
<code>typecheck:safe</code>	TypeScript (non-blocking)

31.8 Security

Script	Command
security:audit	pnpm audit (high severity)
security:fix	pnpm audit fix
check:all	health + system + security + content-boundary
check:system	System-level checks
check:security	audit-ci with config
check:content-boundary	Server/client import boundary
check:deps	Dependency usage check

31.9 Fixes

Script	Command
fix:all	Run ALL fix scripts
fix:windows	Windows path normalization
fix:yaml	YAML formatting
fix:mdx	MDX encoding issues
fix:encoding	UTF-8/BOM issues
fix:prisma	Regenerate Prisma client
fix:imports	Fix broken import paths
fix:build	Fix common build issues
fix:unsafe	Remove unsafe string patterns

31.10 Deploy & Environment

Script	Command
deploy	Validate links → commit → push
deploy:force	Force push (no verify)
env:check	Check environment variables
env:validate	Full env validation
health	System health check
version:check	Check dependency versions
verify:production	Verify live production site

verify:local	Verify local dev server
verify:all	verify:local + check:all

31.11 Assets

Script	Command
assets:optimize	Standard image optimization
assets:enterprise	Premium quality pipeline
assets:enterprise:production	Production AVIF generation
assets:audit	Audit asset integrity
assets:metrics	Asset performance metrics
assets:clean	Remove optimized assets
assets:images:optimize	Image-specific optimization
assets:fonts:generate	Generate font subsets
optimize:premium	Ultra-quality AVIF images

31.12 Audit & Monitoring

Script	Command
audit:links	Verify all internal/external links
audit:email	Email system integrity
audit:pricing	Pricing authority check
audit:checkout	Checkout flow verification
audit:access	Access control consistency
stats	PDF + assets + vault audit combined
stats:full	All statistics
monitor	System monitoring

Chapter 32: Troubleshooting Guide

32.1 Build Failures

MDX Integrity Failure:

```
Error: MDX integrity check failed for content/briefs/example.mdx
```

Fix:

```
pnpm fix:mdx          # Auto-repair encoding
pnpm mdx:sanitize     # Sanitize all MDX
pnpm mdx:integrity    # Re-verify
```

Contentlayer Cache Corruption:

Error: Cannot read properties of undefined (reading 'type')

Fix:

```
pnpm contentlayer:clean # Remove .contentlayer/
pnpm contentlayer:build # Rebuild from scratch
```

Memory Limit (OOM):

FATAL ERROR: Reached heap limit Allocation failed

Fix: Ensure `NODE_OPTIONS=--max-old-space-size=7168` is set. Check for memory leaks in custom scripts.

Build Size Exceeded: Fix:

```
pnpm build:analyze # Identify large chunks
# Review dynamic imports, tree-shaking, excluded_files in netlify.toml
```

32.2 Database Issues

Connection Refused:

Error: Can't reach database server

Fix:

- Verify `DATABASE_URL` in `.env.local`
- For SQLite: ensure `prisma/dev.db` exists (`pnpm db:init`)
- For Neon: check network connectivity and connection string

Migration Drift:

Error: The database schema is not in sync

Fix:

```
pnpm db:push # Force schema sync (dev only)
# OR
pnpm db:reset # Full reset + re-seed (destructive)
```

Seeding Failures:

Error: Unique constraint failed

Fix:

```
pnpm db:reset # Clear all data and re-seed
```

32.3 PDF Generation Failures

LibreOffice Not Found:

```
Error: soffice not found at path
```

Fix (Windows):

```
# Ensure LibreOffice is installed, then in .env.local:  
# The pdf:generate-all script uses:  
# --soffice "C:\Program Files\LibreOffice\program\soffice.com"
```

Puppeteer Chrome Missing:

```
Error: Could not find Chrome
```

Fix:

```
# Docker: PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium  
# Local: npx puppeteer browsers install chrome
```

PDF Size Validation:

```
Error: Generated PDF is 0 bytes
```

Fix:

```
pnpm pdf:repair # Auto-repair empty PDFs
```

32.4 Auth Issues

Secret Mismatch:

```
Error: JWTVerificationFailed
```

Fix: Ensure `NEXTAUTH_SECRET` is identical across all environments. Regenerate if rotated.

Callback URL Mismatch:

```
Error: redirect_uri_mismatch
```

Fix: Add the URL to OAuth provider's allowed redirect URIs. Ensure `NEXTAUTH_URL` matches the actual domain.

Provider Configuration: Fix: Verify all OAuth environment variables are set:

```
GOOGLE_OAUTH_CLIENT_ID=<actual-value>  
GOOGLE_OAUTH_CLIENT_SECRET=<actual-value>
```


32.5 Content Issues

Frontmatter Validation:

```
Error: Invalid frontmatter in content/briefs/example.mdx
```

Fix:

```
pnpm content:validate # Shows exactly which fields are wrong
```

Encoding Issues (Windows):

```
Error: Unexpected token (UTF-8 BOM detected)
```

Fix:

```
pnpm fix:encoding # Removes BOM from all files
```

Windows Path Issues:

```
Error: ENOENT: no such file or directory 'content\briefs\example.mdx'
```

Fix:

```
pnpm fix:windows # Normalizes all paths
```

32.6 Deployment Issues

Bundle Size Too Large:

```
Error: Function exceeds size limit
```

Fix: Review `excluded_files` in `netlify.toml`. Run `pnpm build:analyze` to identify heavy dependencies.

Missing Environment Variables:

```
Error: Required environment variable X not set
```

Fix: Add variable in Netlify dashboard → Site settings → Environment variables. Redeploy.

Function Timeout:

```
Error: Function execution timeout
```

Fix: Optimize the function. Netlify functions have a 26-second timeout (Pro: 60s). Move heavy work to scheduled functions or background functions.

APPENDIX A — ENVIRONMENT VARIABLE REFERENCE

Complete Variable Table

Variable	Category	Required	Default	Description
NODE_ENV	Application	Yes	development	Runtime environment
NEXT_PUBLIC_APP_ENV	Application	Yes	development	Client-side environment
NEXT_PUBLIC_APP_NAME	Application	No	Abraham of London	Display name
NEXT_PUBLIC_APP_URL	Application	Yes	http://localhost:3000	Client-side base URL
NEXT_PUBLIC_SITE_URL	Application	Yes	https://www.abrahamoflondon.org	Canonical URL
SITE_URL	Application	Yes	https://www.abrahamoflondon.org	Server-side site URL
SITE_DOMAIN	Application	No	www.abrahamoflondon.org	Domain without protocol
ALLOWED_ORIGINS	Application	Yes	(see example)	CORS whitelist (comma-separated)
DATABASE_URL	Database	Yes	file:./prisma/dev.db	Primary database connection
DIRECT_URL	Database	Yes	file:./prisma/dev.db	Direct database connection (bypass pooler)
REDIS_URL	Redis	No	redis://127.0.0.1:6379	Redis connection string
REDIS_DISABLED	Redis	No	true	Disable Redis entirely
USE_REDIS	Redis	No	false	Enable Redis features
UPSTASH_REDIS_REST_URL	Redis	No	—	Upstash endpoint URL
UPSTASH_REDIS_REST_TOKEN	Redis	No	—	Upstash token
NEXTAUTH_URL	Auth	Yes	http://localhost:3000	NextAuth URL

NEXTAUTH_SECRET	Auth	Yes	—	JWT sig secret (1 chars)
JWT_SECRET	Auth	Yes	—	Additio JWT sec
JWT_ALGORITHM	Auth	No	HS256	JWT alg
JWT_EXPIRES_IN	Auth	No	30d	Token e
ENCRYPTION_KEY	Auth	Yes	—	General encrypt
CSRF_SECRET	Auth	Yes	—	CSRF to secret
ACCESS_COOKIE_SECRET	Auth	Yes	—	Cookie encrypt secret
SESSION_COOKIE_PREFIX	Auth	No	ao1	Cookie prefix
ADMIN_JWT_SECRET	Admin	Yes	—	Admin- specific
ADMIN_API_KEY	Admin	Yes	—	Admin , authent
ADMIN_SECRET_TOKEN	Admin	Yes	—	Admin t validatio
ADMIN_SECRET	Admin	Yes	—	Admin :
ADMIN_USER_EMAIL	Admin	Yes	—	Bootstr admin e
ADMIN_USER_PASSWORD	Admin	Yes	—	Bootstr admin passwo
ADMIN_ALLOWED_EMAILS	Admin	Yes	—	Admin e whitelis
ADMIN_PASSWORD_HASH	Admin	No	—	Pre-has admin passwo
CRON_SECRET	Cron	Yes	—	Cron er auth
INTERNAL_BYPASS_KEY	Cron	Yes	—	Internal service

AUDIT_EDGE_SECRET	Cron	Yes	—	Audit edge function
AOL_BRAND_NAME	Brand	No	Abraham of London	Brand domain name
AOL_ISSUER_ID	Brand	No	AOL-LOCAL	Issuer identifier
AOL_HASH_SALT	Brand	Yes	—	General hashing
AOL_WATERMARK_ISSUER_MEMBERID	Brand	No	SYSTEM_ARCHIVE	PDF watermark issuer
AOL_SESSION_TTL_DAYS	Brand	No	30	Session lifetime
AOL_TOKENSTORE_BACKEND	Brand	No	postgres	Token store backend
AOL_TOKEN_TTL_HOURS	Brand	No	72	Token lifetime (hours)
SYSTEM_INTEGRITY_SALT	Brand	Yes	—	Vault/PI integrity (min 16)
ANONYMITY_SALT	Brand	Yes	—	Identity anonymity
DENYLIST_PEPPEP	Brand	Yes	—	Denylist pepper
EMAIL_PROVIDER	Email	No	resend	Email service provider
EMAIL_FROM	Email	No	—	From address
EMAIL_REPLY_TO	Email	No	—	Reply-to address
RESEND_API_KEY	Email	Partial	—	Resend key
MAIL_FROM	Email	No	—	Full from header
MAIL_TO	Email	No	—	Default recipient
CONTACT_RECEIVER_EMAIL	Email	No	—	Contact recipient

NOTIFICATION_WEBHOOK_URL	Email	No	—	Webhook notifica
INNER_CIRCLE_STORE	Inner Circle	Yes	postgres	IC data
INNER_CIRCLE_DB_URL	Inner Circle	Yes	—	IC data URL
INNER_CIRCLE_JWT_SECRET	Inner Circle	Yes	—	IC JWT
INNER_CIRCLE_KEY_SECRET	Inner Circle	Yes	—	IC key encrypt
ENTERPRISE_ALIGNMENT_INVITE_SECRET	Enterprise	Yes	—	Enterpri invite H
DIAGNOSTIC_HMAC_SECRET	Enterprise	Yes	—	Diagno: integrity
DIAGNOSTIC_WATERMARK_SECRET	Enterprise	Yes	—	PDF waterm secret
DIAGNOSTIC_STORAGE_PROVIDER	Enterprise	No	local	Storage backend
DIAGNOSTIC_DEFAULT_CURRENCY	Enterprise	No	gbp	Default currenc
OGR_SESSION_SECRET	Sovereign	Yes	—	OGR se signing
OGR_SOVEREIGN_KEY	Sovereign	Yes	—	Sovereign access k
SOVEREIGN_ACCESS_KEY	Sovereign	Yes	—	Sovereign auth
STRIPE_SECRET_KEY	Payments	No	—	Stripe A
STRIPE_WEBHOOK_SECRET	Payments	No	—	Stripe webhook validatio
OPENAI_API_KEY	AI	No	—	OpenAI key
GOOGLE_OAUTH_CLIENT_ID	OAuth	No	—	Google client
GOOGLE_OAUTH_CLIENT_SECRET	OAuth	No	—	Google secret
SLACK_CLIENT_ID	OAuth	No	—	Slack O. client

SLACK_CLIENT_SECRET	OAuth	No	—	Slack O. secret
OAUTH_TOKEN_ENCRYPTION_KEY	OAuth	No	—	Token encrypt (min 32
PDF_OUTPUT_DIR	PDF	No	./public/pdfs	PDF out director
PDF_TEMP_DIR	PDF	No	./temp/pdfs	PDF ten director
PDF_FONTS_DIR	PDF	No	./public/fonts	Font dir
ARTIFACT_ACCESS_SECRET	PDF	Yes	—	Artifact signing
DOWNLOAD_TOKEN_SECRET	PDF	Yes	—	Downlo token H
RATE_LIMIT_MAX_REQUESTS	Security	No	100	Max rec per win
RATE_LIMIT_WINDOW_MS	Security	No	900000	Window duration
ENABLE_ANALYTICS	Feature Flags	No	false	Analytic enablec
ENABLE_PDF_GENERATION	Feature Flags	No	true	PDF generat enablec
ENABLE_EMAIL_NOTIFICATIONS	Feature Flags	No	false	Email se enablec
SKIP_DB	Feature Flags	No	false	Skip da operatio
LOG_LEVEL	Feature Flags	No	warn	Logging verposit
VAULT_CACHE_SECONDS	Feature Flags	No	300	Vault ca TTL
DEBUG_CONTENTLAYER	Development	No	false	Verbose logging
IS_WINDOWS	Development	No	false	Window compat mode
PREMIUM_ASSET_BACKEND	Development	No	local	Asset st backenc

APPENDIX B — DATABASE SCHEMA REFERENCE

Model Summary (124 Models)

The complete Prisma schema contains 124 models across the following domains:

Core Identity & Access

Model	Purpose	Key Relations
User	Platform user	→ Account, Entitlement, AccessKeyUse
Account	OAuth account link	→ User
VerificationToken	Email verification	Standalone
Entitlement	Access grants	→ User
AccessKey	Redeemable access codes	→ AccessKeyUse
AccessKeyUse	Redemption log	→ User, AccessKey
AccessAuditLog	Access attempt log	Standalone
AccessInvite	Invitation tokens	Standalone

Organisation & Enterprise

Model	Purpose	Key Relations
Organisation	Enterprise client	→ Memberships, Campaigns, Invites
AlignmentCampaign	Assessment campaign	→ Organisation, Participants
AlignmentSnapshot	Campaign results	→ Campaign, Organisation
OrganisationInvite	Org join invitation	→ Organisation
OrganisationMembership	User-org binding	→ Organisation, Campaigns
CampaignParticipant	Campaign member	→ Campaign, Membership
EnterpriseAssessment	Individual assessment	→ Campaign
EnterpriseReport	Generated report	→ Campaign

Diagnostics

Model	Purpose	Key Relations
DiagnosticRecord	Diagnostic instance	→ Artifacts, Orders
DiagnosticReportOrder	Payment for report	→ DiagnosticRecord
DiagnosticArtifact	Generated PDF/artifact	→ DiagnosticRecord
DiagnosticRegenerationJob	Report rebuild queue	→ DiagnosticRecord

DiagnosticAuditEvent	Diagnostic audit trail	→ DiagnosticRecord
DiagnosticJourney	Multi-stage diagnostic	→ Evidence, Decisions
DiagnosticEvidenceNode	Evidence data point	→ Journey
DiagnosticDecisionObject	Decision record	→ Journey

Decision Intelligence

Model	Purpose
DecisionRecommendationSession	AI recommendation session
DecisionRecommendationImpression	Content shown to user
DecisionRecommendationClick	User click tracking
DecisionRecommendationConversion	Conversion event
DecisionSessionFollowup	Follow-up actions
DecisionAssetEfficacy	Asset performance metrics
DecisionAssetContextPerformance	Context-specific performance
DecisionSignalRegistry	Signal definitions
DecisionGovernanceAlert	Governance violations
DecisionAssetGovernanceRule	Asset-level rules

Governance & Audit

Model	Purpose	Key Fields
GovernanceLog	Admin action log	action, performedBy, target
SecurityLog	Security events	event, ip, userId
SystemAuditLog	System audit trail	actorType, action, severity
AuditEvent	Generic audit event	type, severity, metadata
RateLimitLog	Rate limit violations	ip, endpoint, count

Inner Circle & Membership

Model	Purpose
InnerCircleMember	IC membership record
InnerCircleKey	IC access key
AdminSession	Admin login session
ApiKey	API key management

ApiLog	API usage logging
MfaSetup	MFA configuration
Session	User session

Content & Assets

Model	Purpose
ContentMetadata	Content piece metadata
Framework	Strategic framework definition
StrategicLink	Framework relationships
ContentRelation	Content cross-references
PrivateAnnotation	Private content notes
CanonEntry	Canon/lexicon entry
StrategicFramework	Framework configuration
PrintAsset	Print-ready asset
DownloadAuditEvent	Download tracking
PremiumDownloadToken	Gated download token
PremiumDownloadAttempt	Download attempt log

Commercial

Model	Purpose
DealFlowSubmission	Inbound deal flow
StrategyInquiry	Strategy inquiry form
StrategyIntake	Intake assessment
RetainerContract	Client retainer
RetainedDecision	Decision under retainer
BillingCustomer	Payment customer
ClientEntitlement	Client-specific access

Enums (34 Total)

AlignmentBand, AlignmentDomain, CorrectionStatus, MemberRole, AccessTier, MemberStatus, ContentType, LinkType, AnnotationPriority, InquiryStatus, StrategyIntakeStatus, DownloadEventType, DownloadDeliveryMode, DownloadContentType, AccessType, AuditSeverity, SecurityEvent, HttpMethod, SessionStatus, KeyStatus, MfaMethod,

Permission, DiagnosticSeverity, DiagnosticLifecycleStatus, DiagnosticReportStatus, DiagnosticArtifactKind, DiagnosticStorageProvider, DiagnosticRegenerationStatus, GovernanceSeverity, GovernanceStatus, UserRole, EntitlementType, EntitlementStatus, AccessKeyStatus, InviteStatus, AuditActorType

APPENDIX C — API ROUTE REFERENCE

Public Routes (No Auth)

Method	Path	Description
GET	/api/root	API root/health
GET	/api/v2/health	Detailed health check
GET	/api/search	Public content search
POST	/api/telemetry/global	Global telemetry events
POST	/api/telemetry/resonance	Content resonance tracking
POST	/api/pulse/submit	Pulse survey submission
GET	/api/stats	Public statistics

Authentication Routes

Method	Path	Description
POST	/api/auth/sovereign	Sovereign auth flow
POST	/api/sovereign/auth	Sovereign login
POST	/api/sovereign/logout	Sovereign logout
GET	/api/sovereign/history	Auth history
POST	/api/admin/dev-login	Dev-only admin login

Inner Circle

Method	Path	Description
POST	/api/inner-circle/verify	Verify IC membership
POST	/api/inner-circle/issue	Issue IC credential
GET	/api/inner-circle/admin/export	Export IC data (admin)

Enterprise & Alignment

Method	Path	Description
GET/POST	/api/alignment/enterprise	Enterprise alignment

POST	/api/alignment/enterprise/organisations	Create organisation
GET/POST	/api/alignment/enterprise/campaigns	Campaign CRUD
GET/PUT	/api/alignment/enterprise/campaigns/[id]	Single campaign
POST	/api/alignment/enterprise/campaigns/[id]/close	Close campaign
POST	/api/alignment/enterprise/campaigns/[id]/invite	Invite participant
GET	/api/alignment/enterprise/campaigns/[id]/report	Campaign report
POST	/api/alignment/enterprise/campaigns/[id]/aggregate	Aggregate results
POST	/api/alignment/enterprise/respond/[token]	Submit response
GET	/api/alignment/enterprise/assessments	List assessments

Team Assessment

Method	Path	Description
POST	/api/team-assessment/campaign/create	Create team campaign
POST	/api/team-assessment/campaign/[id]/invites	Send invites
POST	/api/team-assessment/campaign/[id]/close	Close assessment
GET	/api/team-assessment/campaign/[id]/aggregate	Team aggregate
GET	/api/team-assessment/campaign/[id]/status	Campaign status
POST	/api/team-assessment/respond/[token]	Submit response

Diagnostics

Method	Path	Description
POST	/api/diagnostics/campaigns/[id]/aggregate	Diagnostic aggregate
POST	/api/diagnostics/outcomes/verify	Verify outcomes
POST	/api/diagnostics/reentry	Re-entry flow
GET	/api/diagnostics/longitudinal	Longitudinal comparison

Decision Intelligence

Method	Path	Description
POST	/api/decision/guidance	Decision guidance AI
GET	/api/decision/metadata-audit	Metadata audit
POST	/api/interpret	Content interpretation

Strategy Room

Method	Path	Description
POST	/api/strategy-room/session/init	Initialize session
POST	/api/strategy-room/session/impression	Log impression
POST	/api/strategy-room/session/click	Log click
POST	/api/strategy-room/session/conversion	Log conversion
POST	/api/strategy-room/session/followup	Follow-up action
GET	/api/strategy-room/results	Session results
POST	/api/strategy-room/conversion	Conversion tracking
GET	/api/strategy-room/execution/[id]	Execution session

Downloads & Vault

Method	Path	Description
GET	/api/download/[token]	Token-gated download
GET	/api/downloads/[slug]	Slug-based download
GET	/api/vault/status	Vault status
GET	/api/vault/[...slug]	Vault content access

Reports & Export

Method	Path	Description
GET	/api/campaigns/[id]/report	Campaign report
GET	/api/campaigns/[id]/report/json	JSON export
GET	/api/campaigns/[id]/report/pdf	PDF export
GET	/api/sovereign/report	Sovereign report
GET	/api/purpose-alignment/report	PA report
GET	/api/purpose-alignment/report/[assessmentId]	Single PA report

Admin Routes (Founder/Admin Only)

Method	Path	Description
GET/POST	/api/admin/campaigns	Campaign management
GET/PUT	/api/admin/campaigns/[id]	Single campaign admin

GET	/api/admin/campaigns/[id]/report	Admin report view
GET	/api/admin/campaigns/[id]/report/pdf	Admin PDF report
GET	/api/admin/campaigns/[id]/report/export-json	Admin JSON export
GET	/api/admin/campaigns/[id]/report-data	Raw report data
GET	/api/admin/decision/efficacy	Decision efficacy
GET	/api/admin/decision/contextual-efficacy	Contextual efficacy
GET	/api/admin/decision/contextual-ranking	Context ranking
GET	/api/admin/decision/governance	Governance dashboard
GET	/api/admin/decision/performance	Performance metrics
GET	/api/admin/decision/signal-registry	Signal registry
POST	/api/admin/decision/rebuild-efficacy	Rebuild efficacy
POST	/api/admin/decision/rebuild-contextual-efficacy	Rebuild context
POST	/api/admin/decision/rebuild-governance-alerts	Rebuild alerts
POST	/api/admin/decision/rebuild-performance	Rebuild performance
GET	/api/admin/decision-intelligence	DI dashboard
GET	/api/admin/commercial	Commercial metrics
GET	/api/admin/positioning	Market positioning

Other Routes

Method	Path	Description
POST	/api/audit/log	Submit audit log
POST	/api/audit/submit	Audit form submission
POST	/api/audit/[id]/submit	Specific audit submission
POST	/api/constitutional/appeal	Constitutional appeal
GET	/api/constitutional/audit	Constitutional audit
GET	/api/constitutional/export	Export constitutional data
POST	/api/cron/snapshot	Cron snapshot trigger
GET	/api/editorials/[slug]	Editorial content
GET	/api/entitlements	User entitlements
POST	/api/executive-reporting/entitlements	Exec report access

GET	/api/executive-reporting/export/pdf	Exec PDF export
GET	/api/executive-reporting/export/boardroom-pdf	Boardroom PDF
POST	/api/executive-reporting/export/intervention	Intervention export
POST	/api/interactions/toggle	Toggle interaction
POST	/api/leads/fuse	Lead fusion
GET	/api/predictive/insights/[campaignId]	Predictive insights
POST	/api/premium/forensics/attribution	Attribution forensics
GET	/api/purpose-alignment/assessments	PA assessments
GET/PUT	/api/purpose-alignment/reminders/preferences	Reminder prefs
POST	/api/purpose-alignment/reminders/preferences/run	Run reminders
GET	/api/team/respondents/[token]	Team respondent info
POST	/api/checkout	Payment checkout
POST	/api/analytics/journey	Journey analytics
POST	/api/live/constitutional-posture	Live posture check
GET	/api/v2/users	User management

APPENDIX D — REVISION HISTORY

Version	Date	Author	Changes
1.0	April 2026	Engineering Lead	Initial release — Part C

Amendment Procedure

1. **Proposal** — Submit change request to Engineering Lead with rationale
2. **Review** — Founder reviews all changes to security or architecture sections
3. **Testing** — Changes validated against running system
4. **Approval** — Founder sign-off required for any security/access modifications
5. **Publication** — Version incremented, date updated, changelog entry added
6. **Communication** — All engineering staff notified of changes

—— **KEY PRINCIPLE**

This manual is a living document but NOT a wiki. Changes require formal review and approval. The manual reflects the system as it IS, not as someone wishes it were. If the system changes, the manual is updated. If the manual says something the system does not do, the system is wrong.

CLOSING DECLARATION

This document — Engineering Manual Part C — together with Parts A and B, constitutes the complete, authoritative, and binding engineering reference for the Abraham of London platform.

Every architectural decision, security control, deployment configuration, and operational procedure documented herein is the product of deliberate design. They are not suggestions. They are not starting points for discussion. They are the standard.

Any engineer working on this platform — internal, contracted, or automated — is bound by the protocols, conventions, and prohibitions contained in this manual. Deviation without explicit written authorization from the Founder constitutes a breach of engineering discipline.

⚠ PROHIBITION

No AI assistant, code generator, or automated tool may override, ignore, or "improve upon" the standards in this manual. If an AI suggests a pattern that contradicts this document, this document wins. Always. The manual is the final authority on how this system is built, deployed, secured, and operated.

This is not a 5-year document. This is a 100-year document. The tools will change. The frameworks will evolve. But the principles of disciplined engineering — validate inputs, enforce access, log everything, fail safely, deploy with confidence — are permanent.

Build accordingly.

END OF ENGINEERING MANUAL — PART C

Abraham of London — Decision Authority Infrastructure Effective April 2026